04820KA001 Mono Canada Slitter 903 1029

Author: Michou
Created: 10/29/2026 10:38:50
Last Modified: 10/29/2026 10:38:50
Comment: Modified to Connect 2nd Diameter Sens

Sysmac Studio Module Version: 1.60.0.64010

Table of Contents

04820KA001 Mono Canada Slitter 903 1029	3
1.PLC	3
1-8.POUs	3
1-8-1.Programs	3
1-8-1-1.TaglMonoTRM	3
1-8-1-1-1.Variables	3
1-8-1-1-2.General	12
1-8-1-1-3.Motor_Manager	16
1-8-1-1-4.OverRider_Manager	24
1-8-1-1-5.Console	27
1-8-1-1-6.Alarms	30
1-8-1-1-7.Output_Manager	35
1-8-1-1-8.AutoStop_Control	38
1-9.Data	52
1-9-1.Data Types	52
1-9-2.Global Variables	53

1.PLC

1-8.POUs

1-8-1.Programs

1-8-1-1.TaglMonoTRM

1-8-1-1-1.Variables

Name	Data Type	Initial Value	AT	Retain	Constant	Comment
VAR						
SpdDelta	REAL			False	False	Minimum or maximum speed SpdDeltaerence
SpdDeltaMin	REAL			False	False	Minimum speed difference
SpdDeltaMax	REAL			False	False	Maximum speed SpdDeltaerence
SoftStopMem	BOOL			False	False	v6r: soft stop edge memory
SoftStopRatio	REAL			False	False	v6r: InvRef counts per m/min, frozen at the soft stop start
Clock100ms	BOOL			False	False	100 ms clock
MemPos	DINT			False	False	Previous encoder count
MatChangeCnt	INT			False	False	v6s: consecutive learning windows far from RwMat_K
DeltaPulse	DINT			False	False	Encoder count difference
SlowMem	BOOL			False	False	SlowDown edge memory
DecelDiam	REAL			False	False	Rewinder diameter used by the decel model (mm, min 100)
TeachSwap	REAL			False	False	Temporary for UNDO swap
SlowStartDiam	REAL			False	False	Diameter at slow-down start (mm)
SoftStartTicks	INT			False	False	S-curve start: 4 ms ticks since start
SoftStartK	REAL			False	False	S-curve start: accel factor 0.05..1
BrakeHold	BOOL			False	False	Keep T-one in auto during a normal stop (O_Brake)
StopNow	BOOL			False	False	Immediate stop request (replaces I_Stop_PB in General rung 9)
StopPB_Mem	BOOL			False	False	STOP button edge memory
T_StopLong	TON			False	False	STOP held 1 s = immediate stop
T_SoftStop	TON			False	False	Soft stop timeout (120 s)
BrakeHoldTicks	INT			False	False	BrakeHold duration after stop (100 ms ticks, max 150)
StopStartDiam	REAL			False	False	Diameter at stop command (mm)
DecelK_Meas	REAL			False	False	Measured decel per mm of diameter
SlowMeasActive	BOOL			False	False	Slow-down measurement running
TeachTicks	INT			False	False	TEACH CORE stable time (100 ms ticks)
SlowStartPos	DINT			False	False	Encoder at slow-down start
SlowStartSpd_Pps	REAL			False	False	Speed at slow-down start (pulses/s)
SlowMeas_Pls	REAL			False	False	Measured slow-down length (pulses)
SlowDecelMeas_mm	REAL			False	False	Measured slow-down decel (mm/s2)
SlowOneRamp	BOOL			False	False	Slow-down uses one ramp (manual or learned)
Velocity	REAL			False	False	Speed in [mm/s]
Alarm_Array	ARRAY[0..32] OF BOOL			False	False	Alarm
T_PlcOn	TON			False	False	PLC active timeout

ControlVoltage	BOOL			False	False	Auxiliaries OK
StartMotor_Cmd	BOOL			False	False	Motor start command
HubExtractEnable	BOOL			False	False	Hub extraction enable command
Brake_Cmd	BOOL			False	False	Brake command
T_StopVent	TOF			False	False	Fans stop timer
TimStopVen	TIME			False	False	Fans stop timeout
MotorCooling	BOOL			False	False	Fans command
OverRider_Auto	BOOL			False	False	OverRider in automatic
OverRider_Cont	BOOL			False	False	OverRider in continuous
OverRider_Lock	BOOL			False	False	OverRider lock command
GSM_LearnFault	BOOL			False	False	Paper [Grams / Square Meter] LearnFailCnt ≥ RwLearnFailMax
TickCnt	INT			False	False	100 ms tick scan counter (25 x 4 ms)
RunWasJog	BOOL			False	False	Run used jog / manual / slow button (2nd ramp)
StopLearnValid	BOOL			False	False	This stop may be learned
SpdBand_Pps	REAL			False	False	Speed band of the trim ramp (pulses/s)
TrimDecel_Pps2	REAL			False	False	Trim ramp decel (pulses/s2)
SpdKnee_Pps	REAL			False	False	Speed where coarse ramp hands over to trim ramp (pulses/s)
OverRider_RaiseUp	BOOL			False	False	OverRider raise command
Pot1_Mem	INT			False	False	Work potentiometer value
Clock10ms	BOOL			False	False	10 ms clock
VelSet_mms	REAL			False	False	Set speed in mm/s
Vfd_SP_Modif	REAL			False	False	Inverter speed modifier
RwAutoLimAct	BOOL			False	False	v6y: rewinder AUTO speed limit active (taken over at standstill)
UwTeachTicks	INT			False	False	Unwinder TEACH CORE samples (100 ms)
T_PosPresOff	TON			False	False	OverRider photocell OFF timer
T_PosPresOn	TON			False	False	OverRider photocell ON timer
TimPosPresOff	TIME			False	False	OverRider photocell OFF timeout
TimPosPresOn	TIME			False	False	OverRider photocell ON timeout
RollEnd_Active	BOOL			False	False	Roll diameter photocell intercepted
Output_Ratio	REAL			False	False	Analog output ratio
Input_Ratio	REAL			False	False	Analog input ratio
T_StartLow	TON			False	False	Slow start timer
T_UwDiamDebounce	TON			False	False	Roll diameter photocell Debounce timer 500mS
UwDiamReached	BOOL			False	False	Roll diameter reached
Index_A	INT			False	False	
AlarmStrobe	BOOL			False	False	Alarm message display strobe
T_Display	TON			False	False	Alarm display timer
RangeOK	BOOL			False	False	Range reached
Vel_Max_mms	REAL			False	False	Maximum speed in mm/s
Slow_Inhibit	BOOL			False	False	Inhibit slow button
Decel_Ramp	REAL			False	False	Deceleration ramp
Accel_Ramp	REAL			False	False	Acceleration ramp
Pot2_Mem	INT			False	False	Final potentiometer value
Current_mm	REAL			False	False	Current millimetres
EncoderPulse_to_mm	REAL			False	False	mm per encoder pulse
SlowDown	BOOL			False	False	Slow down

Stop	BOOL			False	False	Stop
T_EncoderStill	TON			False	False	(?) Encoder stopped timer
Analog_Res	UINT			True	False	Analog board resolution
Mt_To_mm	REAL			False	False	m/min to mm/s factor
Sec_to_Min	REAL			False	False	mm/s to m/min factor
Pot1_Word	WORD			False	False	(?) Pot 1 raw word
Pot1_WordSW	WORD			False	False	(?) Pot 1 raw word, byte-swapped
Pot2_Word	WORD			False	False	(?) Pot 2 raw word
Pot2_WordSW	WORD			False	False	(?) Pot 2 raw word, byte-swapped
InvRef2	UINT			False	False	(?) Inverter reference 2
InvRefSW	WORD			False	False	(?) Inverter reference, byte-swapped
InvRefWW	WORD			False	False	(?) Inverter reference, word
Init_Ramp	BOOL			True	False	(?) Ramp initialisation
SlowByDiam	BOOL			False	False	(?) Slow down by diameter
UwStopByDiam	BOOL			False	False	Stop by diameter
T_Screen_Delay	TON			False	False	(?) Start page timer
T_PulStop	TON			False	False	(?) Stop button timer
Reading_Max	REAL	2000		True	False	Maximum sensor reading (full scale)
DeadZone	REAL	60		True	False	Sensor dead zone
In_UwDiameter	REAL			True	False	UnWinder Diameter Analog input voltage
T_Sample	TON			False	False	100 ms tick timer
Tick	BOOL			False	False	100 ms tick
MmPerPulse	REAL			False	False	mm per encoder pulse
RwDistRaw_mm	REAL			False	False	Rewinder Unfiltered distance (mm)
RwFiltInit	BOOL			False	False	Rewinder Filter initialised
SpdInit	BOOL			False	False	Speed init
EncPrev	DINT			False	False	Encoder at previous tick
SpdAct_Pps	REAL			False	False	Actual Speed (pulses/s)
LearnInit	BOOL			False	False	Learn initialised
LearnWinRunning	BOOL			False	False	Motor ran whole window
LearnWinTicks	INT			False	False	Learn window counter
Encoder_Last	DINT			False	False	Encoder at window start
Diameter_Last	REAL			False	False	Diameter at window start
LearnWin_Pls	DINT			False	False	Pulses in window
LearnWinDiamDelta	REAL			False	False	Diameter change in window
MatK_Window	REAL			False	False	Material factor this window
MotorRunMem	BOOL			False	False	Motor start edge memory
StopMeasActive	BOOL			False	False	Stop measurement running
StopStartPos	DINT			False	False	Encoder at stop command
StopStartSpd_Pps	REAL			False	False	Speed at stop command (pulses/s)
StandstillTicks	INT			False	False	Standstill tick counter
LearnWinOk	BOOL			False	False	Learning window gave a valid result
ThickWin_mm	REAL			False	False	Paper thickness this window (mm)
LearnFailCnt	INT			False	False	Consecutive failed learning windows
StopLearnOk	BOOL			False	False	Stop measurement gave a valid result
StopDecelMeas_mm	REAL			False	False	Measured stop decel (mm/s2)

	StopMeas_Pls	REAL			False	False	Measured stop length (pulses)
	StopDecelMeas	REAL			False	False	Measured stop decel (pulses/s2)
	StopLen_Pls	REAL			False	False	Stop length (pulses)
	StopLenFinal_Pls	REAL			False	False	Stop length from slow speed (pulses)
	RwSlowLen_Pls	REAL			False	False	Rewinder Slow-down length (pulses)
	SlowDecel_Pps2	REAL			False	False	Slow-down decel (pulses/s2)
	SpdFinal_Pps	REAL			False	False	Final speed (pulses/s)
	RwRemain_Pls	REAL			False	False	Rewinder Remaining length to target (pulses)
	LearnWinStartPos	DINT			False	False	Rewinder Position Window 0
	LearnWinStartDiam	REAL			False	False	Rewinder Diameter Window 0
	T_ByPassProtOff	TOF			False	False	Delay Off Timer
	UwEnMem	BOOL			False	False	Edge memory UwStopEnable
	RwEnMem	BOOL			False	False	Edge memory RwEnable
	SpdRef_Pps	REAL			False	False	Reference speed MAX (actual,setpoint) (pulses/s)
	SpdWork_Pps	REAL			False	False	Work speed setpoint (pulses/s)
	SlowLenBase_Pls	REAL			False	False	Slow-down length without margin (pulses)
	UwSlowLen_Pls	REAL			False	False	Unwinder slow-down length (pulses)
	UwMarginUsed_m	REAL			False	False	Unwinder margin at slow speed (m)
	SqrtArg	REAL			False	False	Square root argument
	AutoSlowCmd	BOOL			False	False	Automatic Slow Command
	AutoStopCmd	BOOL			False	False	Automatic Stop Command
	RwLastValidDiam	REAL			False	False	Rewinder Last valid diameter (mm)
	RwRemainMax_Pls	REAL			False	False	Rewinder Remaining length to max diameter (pulses)
	UnWinder_Empty	BOOL			False	False	UnWinder Empty
	StopFailCnt	INT			False	False	Consecutive failed stop measurements
	UwTeachSum	REAL			False	False	Unwinder TEACH CORE sonar sum (mm)
	T_MatReset	TON			False	False	v6s: RwMatReset stays ON 3 s
	RwAutoMaxSpdAct	REAL			False	False	v6y: active rewinder AUTO max speed (taken over at standstill)
	UwEmptyMem	BOOL			False	False	v6x: UnWinder_Empty edge memory
	LearnWinLen_Pls	REAL			False	False	v6w: learning window length in encoder pulses
VAR_EXTERNAL							
	VelSet	REAL				False	Set speed
	OverSpeed	BOOL				False	Speed too high
	SpdDelta_Percent	REAL				False	Speed difference percentage
	Encoder_Pos	DINT				False	Encoder counter
	Velocity_M_min	REAL				False	Speed in m/min
	Current_Metre	REAL				False	Current metres
	I_RsetCnt	BOOL				False	Metre counter reset button
	WorkSpd_Max_Set	REAL				False	HMI request: work pot full scale (m/min), applied only with motor stopped

FinalSpd_Max_Set	REAL				False	HMI request: final pot full scale (m/min), applied only with motor stopped
SpdMax_Pending	BOOL				False	New pot range waiting for the motor to stop
RwAxisDist_Prev	REAL				False	Rewinder: RwAxisDist before the last TEACH (for UNDO)
UwCoreDist_Prev	REAL				False	Unwinder: UwCoreDistance before the last TEACH (for UNDO)
RwTeachUndo	BOOL				False	HMI: UNDO rewinder TEACH (PLC resets it)
UwTeachUndo	BOOL				False	HMI: UNDO unwinder TEACH (PLC resets it)
RwLearnReset	BOOL				False	HMI / Watch: clear learned decels RwStopDecelK, RwStopDecel, RwSlowDecelK (PLC resets it)
UnderSpeed	BOOL				False	Speed too low
I_ControlVoltage	BOOL				False	Auxiliaries ON
I_OverRider_PH	BOOL				False	Roll diameter photocell
RwCoreDiameter	REAL				False	Rewinder core outside diameter for TEACH CORE (mm)
UwTeachState	INT				False	Unwinder TEACH CORE: 0 idle 1 teaching 2 done 3 refused
SoftStop	BOOL				False	Soft stop in progress (HMI)
SoftStopEnable	BOOL				False	STOP button = soft stop (False = immediate as before)
SoftStopEndSpd	REAL				False	Soft stop: run command drops below this speed (m/min)
SoftStopDecel	REAL				False	Soft stop decel (mm/s2), 0 = DecelCoarse
RwTeachCore	BOOL				False	HMI: TEACH CORE request (PLC resets it)
RwTeachState	INT				False	TEACH CORE: 0 idle 1 teaching 2 done 3 refused
O_MotorStart	BOOL				False	Motor start
RwSlowDecelK	REAL				False	Learned slow-down decel per mm of rewinder diameter (mm/s2 per mm)
RwStopDecelK	REAL				False	Learned stop decel per mm of rewinder diameter (mm/s2 per mm)
I_OR_Position_Sens	BOOL				False	Rider roll position sensor
WorkSpd_Max	REAL				False	Work speed pot full scale (m/min), 50..VelMax
FinalSpd_Max	REAL				False	Final speed pot full scale (m/min), 5..WorkSpd_Max
I_OR_Pressure_Sens	BOOL				False	Rider roll pressure switch
I_OR_Auto_OS	BOOL				False	Rider roll in automatic
I_OR_Cont_OS	BOOL				False	Rider roll in continuous
I_Start_PB	BOOL				False	Start button
I_SafeGuard_Closed	BOOL				False	Side doors closed
I_Motor_Thermal	BOOL				False	Motor thermal switch
I_FanTemp_Ok	BOOL				False	Motor (fan) thermal OK
I_Stop_PB	BOOL				False	Stop button
I_Hub_Position	BOOL				False	Hub/chuck in extraction position

I_FrontGuard_Closed	BOOL				False	Front guard closed
RwMaxSlowFallback	REAL				False	Rewinder max: slow-down diameter while K not learned (mm)
RwMaxSlowAtDiam	REAL				False	Rewinder max: slow-down starts at diameter (mm)
O_Alarm_Array	BOOL				False	Alarm lamp
O_RwUnload	BOOL				False	Unloaders enable - opening
I_Prot_ByPass	BOOL				False	Guards-bypassed selector switch
O_Prot_ByPass_Lamp	BOOL				False	Guards bypassed lamp
O_Brake	BOOL				False	Brake command
Fan_Timeout	REAL				False	Fans stop timeout (from panel)
O_CoolingFan	BOOL				False	Brake and motor fans
O_OR_Auto_Lamp	BOOL				False	Rider roll auto/continuous lamp
O_OverRider_Up	BOOL				False	Raise rider roll
O_OverRider_Hold	BOOL				False	Lock rider roll
Fix10Vdc	INT				False	Fixed 10 Vdc generator
InvRef	INT				False	Inverter speed reference
Encoder_On	BOOL				False	Encoder enable
Encoder_Reset	BOOL				False	Encoder reset
OverRider_TimeOFF	REAL				False	Rider roll photocell OFF timeout (panel)
OverRider_TimeON	REAL				False	Rider roll photocell ON timeout (panel)
VelMax	REAL				False	Maximum speed
I_LowSpd_OS	BOOL				False	Slow speed button
O_Motor_Lamp	BOOL				False	Motor ON lamp
AlarmOn	BOOL				False	Alarm active
BuzzerOn	BOOL				False	Panel buzzer command
I_VFD_Alarm	BOOL				False	Inverter in alarm
Msg	INT				False	Message
Index_B	INT				False	Active alarm index
MotStop	BOOL				False	Motor stopped
Clock1s	BOOL				False	1 s clock
ManMode	BOOL				False	Manual mode
Speed_Man_SP	INT				False	% speed in manual
Counter_Reset	BOOL				False	Panel counter reset button
I_FrontGuard_Open	BOOL				False	Front guard open
RwUnload_Enable	BOOL				False	Unloaders enable - closing
JogSpd_SP	REAL				False	Speed in manual
O_AccDec2	BOOL				False	Second acc/dec ramp
AccelTrim	REAL				False	Acceleration coefficient in run
DecelTrim	REAL				False	Deceleration coefficient in run
DecelCoarse	REAL				False	Acceleration coefficient at start (orig. comment; name suggests decel)
AccelCoarse	REAL				False	Deceleration coefficient at start (orig. comment; name suggests accel)
Speed_Display	REAL				False	Set speed (displayed)
O_AcousticSignal	BOOL				False	Horn / acoustic signal
Pot1_Work_Velo	INT				False	Potentiometer 1 value - Work spe

city						ed -
O_FrontGuard_ByPass	BOOL				False	Front guard bypass
Pot2_Slow_Velocity	INT				False	Potentiometer 2 value - Final speed -
LowSpd_Display	REAL				False	Slow-down speed (displayed)
Encoder_Pulse	INT				False	Encoder pulses
LowSpd_Man_SP	INT				False	% slow-down speed in manual
RwSlowDecelLrn	REAL				False	Learned real slow-down decel (mm/s2) - 0 = not learned
UwSlowMargin_m	REAL				False	Slow-down metres
UwStop_At_Meter	REAL				False	Total metres
I_AutoStop_OS	BOOL				False	Stop on metre counter
I_RollEnd_SlowDown	BOOL				False	Slow down at end of roll
I_SlowSens_PH	BOOL				False	Slow-down photocell
I_WebBreak_Monitor	BOOL				False	Paper break monitoring ON
I_WebBreak_PH	BOOL				False	Paper break photocell
I_Vacuum_OK	BOOL				False	Extractor fan thermal OK
CountReset_Req	BOOL				False	Metre counter reset request
Psw	INT				False	Password
I_RsetAlarm	BOOL				False	Alarm reset
Fix10Vdc_1	UINT				False	Fixed 10 Vdc generator
EncoderWheel_Diam	REAL				False	Encoder wheel diameter
AlarmCodeSV	ARRAY[0..32] OF BOOL				False	Panel alarms
SafeGuard_ByPass	BOOL				False	Guards bypassed
FrontGuard_Closed	BOOL				False	Front guard closed
Core_Measure	BOOL				False	(?) Stored core value
UwDiam_Measure	REAL				False	Measured distance
UwCoreDistance	REAL				False	(?) Core measurement
RwSlowFallbackMm	REAL				False	
AI_Diam_Unwinder	INT				False	Analog value - parent roll measurement
RwMatChangePct	REAL				False	
UwCoreDiameter	REAL				False	(?) Shaft diameter
Measure_ON	BOOL				False	(?) Measurement enabled
UwDiam_Act	REAL				False	(?) Measured roll diameter
UwDiamStop	REAL				False	(?) Stop diameter
UwSlowDown_Diam	REAL				False	(?) Slow-down diameter
Alarm_Reset	BOOL				False	(?) Alarm reset from panel
HMI_Screen	ARRAY[0..1] OF DINT				False	HMI panel page
Psw22	BOOL				False	(?) Password 22
RwMatReset	BOOL				False	
Psw44	BOOL				False	(?) Password 44
I_Roll_OK	BOOL				False	(?) Roll OK
I_RollEnd_PH	BOOL				False	(?) End-of-roll photocell

AI_Diam_Rewinder	INT				False	Rewinder DT35 raw - I/O Map Unit2 Ch2
RwStopEnable	BOOL				False	Rewinder diameter stop enabled
RwTargetDiam	REAL				False	Rewinder final diameter (mm)
RwUseSlow	BOOL				False	Slow down before target
RwSlowMargin_m	REAL				False	Metres at slow speed before stop
RwSlowDecel	REAL				False	Slow-down decel mm/s2 - 0 = auto from DecStart
RwResetHyst	REAL				False	Reset requests below target - this (mm)
RwDist0V	REAL				False	DT35 distance at 0 V (mm)
RwDist10V	REAL				False	DT35 distance at 10 V (mm)
RwRawMin	INT				False	Raw below = sensor fault
RwRawMax	INT				False	Raw above = sensor fault
RwStopOnFault	BOOL				False	Stop on sensor fault
RwAxisDist	REAL				False	Sensor to shaft centre (mm) - teach
RwFilter_K	REAL				False	Distance filter per 100 ms (0..1)
RwWinTicks	INT				False	Material learn window (100 = 10 s)
RwMinPulses	DINT				False	Min pulses in window to learn
RwLearn_K	REAL				False	Material factor filter weight
RwMat_K	REAL				False	Learned material factor
RwLearnStop	BOOL				False	Learn stop decel on every stop
RwMinLearnSpd	REAL				False	Min speed to learn a stop (m/min)
RwStopDecel	REAL				False	Learned stop decel (pulses/s2)
RwTreact	REAL				False	Stop reaction time (s)
RwTstopDef	REAL				False	Stop time until decel learned (s)
RwTrim	REAL				False	Stop length trim (>1 = earlier)
RwDist	REAL				False	Filtered distance (mm)
RwDiam	REAL				False	Rewinder diameter (mm)
RwSpeed	REAL				False	Line speed from encoder (m/min)
RwRemain_m	REAL				False	Metres to target diameter
RwStopLen_m	REAL				False	Stopping length now (m)
RwSlowLen_m	REAL				False	Slow-down length now (m)
RwStopReq	BOOL				False	Rewinder diameter STOP request
RwSlowReq	BOOL				False	Rewinder diameter SLOW request
RwSensFault	BOOL				False	Rewinder DT35 sensor fault
UwStopEnable	BOOL				False	Unwinder metre-counter stop ON (HMI)
SlowAtDisp	REAL				False	HMI: slow-down starts at (m or mm, see SlowAtMode)
SlowAtMode	INT				False	HMI: 0=none 1=unwinder [m] 2=rewinder [mm]
SlowAtShow	BOOL				False	HMI: show slow-down display
UwSlow_at_Metre	REAL				False	Unwinder: slow-down starts at (m)
UwSlowReq	BOOL				False	Unwinder metre slow-down request (latched)
RwSlowAtDiam	REAL				False	Rewinder: slow-down starts at diameter (mm)
RwStopAtDiam	REAL				False	Rewinder: stop command at diameter (mm)
RwMaxDiam	REAL				False	Rewinder max diameter protection (mm)
RwMaxStop	BOOL				False	Rewinder max diameter STOP (latched)

	RwMaxSlow	BOOL				False	Rewinder max diameter SLOW (latched)
	UwEmptyHyst	REAL				False	
	RwThickMin_mm	REAL				False	Learning: min plausible paper thickness (mm)
	RwThickMax_mm	REAL				False	Learning: max plausible paper thickness (mm)
	RwDecelMin_mm	REAL				False	Learning: min plausible stop decel (mm/s2)
	RwDecelMax_mm	REAL				False	Learning: max plausible stop decel (mm/s2)
	RwLearnFailMax	INT				False	Learning: failed windows before alarm (12 = 2 min)
	RwStopFailMax	INT				False	Learning: failed stops before alarm
	GSM_Learned_OK	BOOL				False	Rewinder learning fault (alarm 16)
	RwThick_mm	REAL				False	Learned paper thickness (mm)
	Psw33	BOOL				False	
	RwAutoMaxSpd	REAL				False	
	RwWinLen_m	REAL				False	
	RwWinGrow_mm	REAL				False	
	GsmAsk	BOOL				False	
	GsmAskYes	BOOL				False	

1-8-1-1-2.General

0 Remote 10.146.233.11 ... 1s clock.



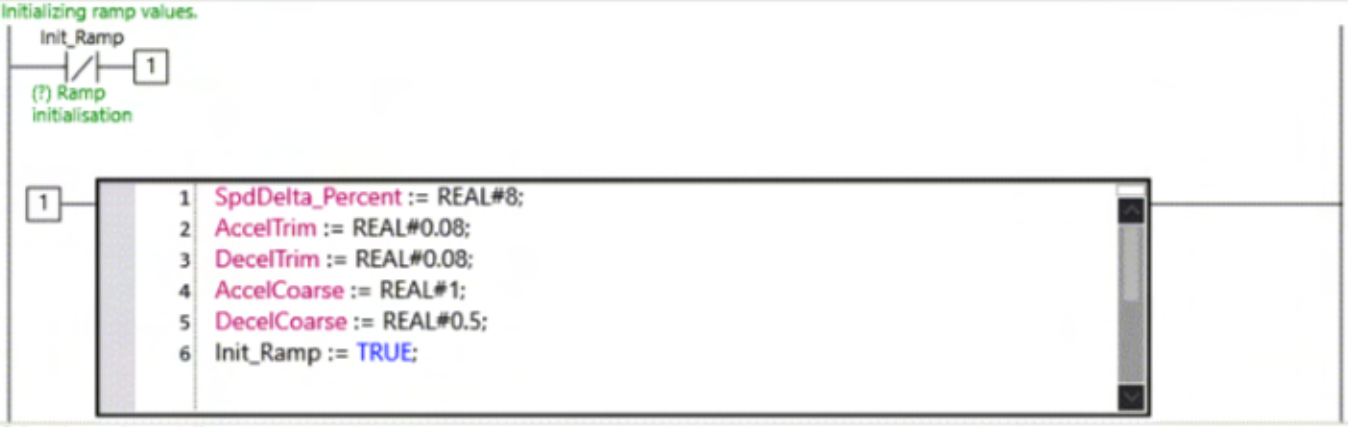
1 100ms clock.



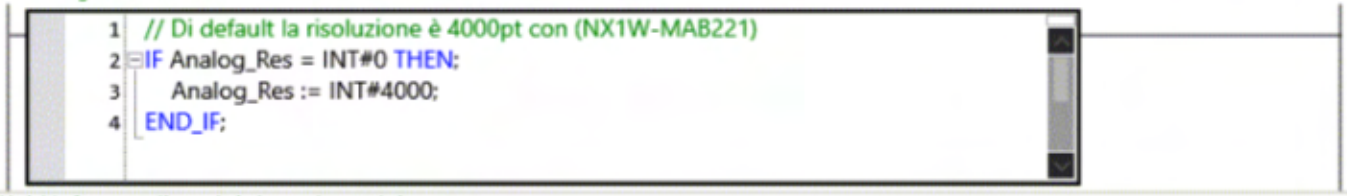
2 10ms clock.



3 Initializing ramp values.

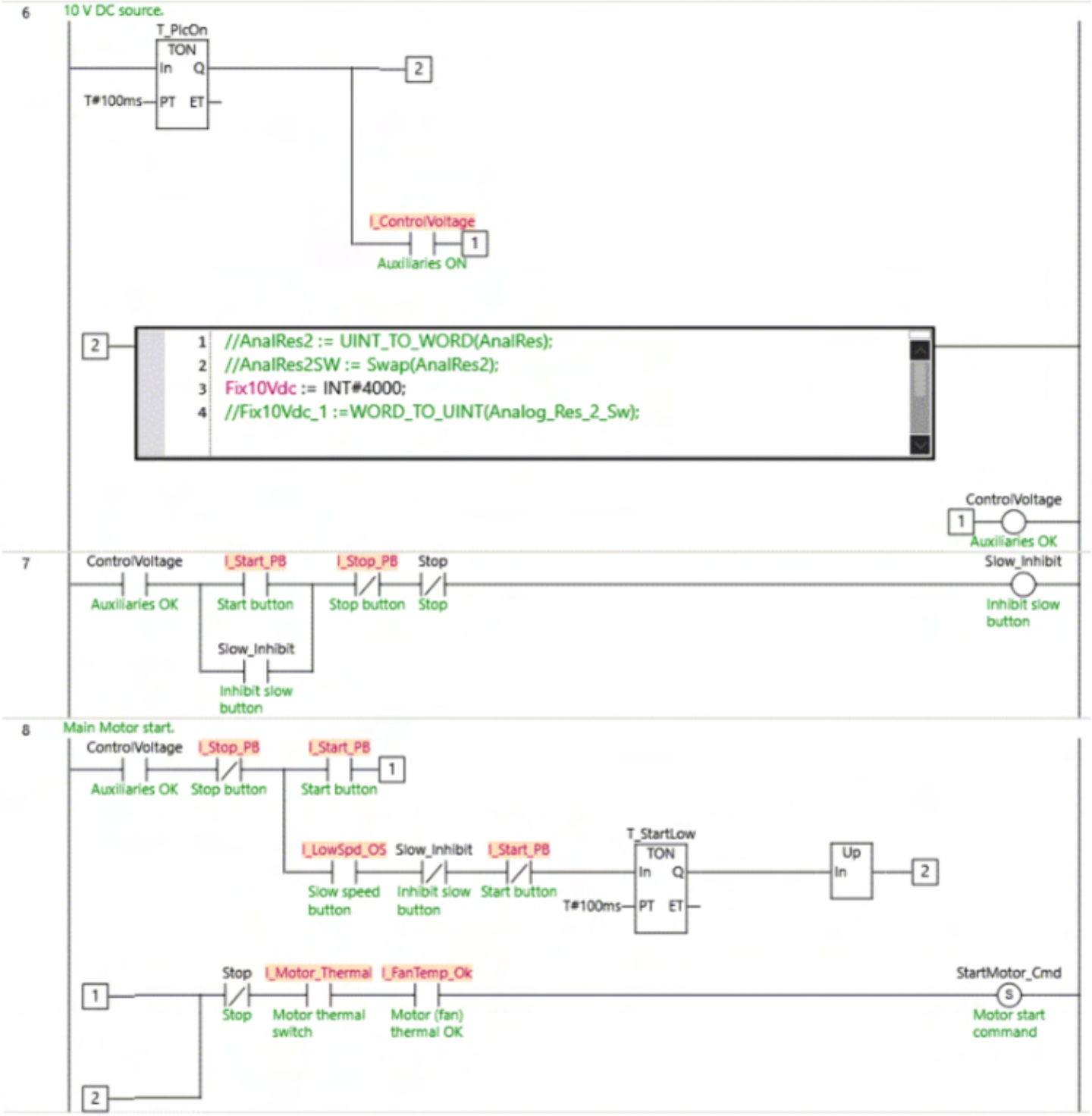


4 Set analog scale resolution.



5 Variable for converting m/min to mm/sec and vice versa.




6
10 V DC source.
T_PlcOn
TON
In Q
T#100ms PT ET
2
I_ControlVoltage
1
Auxiliaries ON
2
1 //AnalRes2 := UINT_TO_WORD(AnalRes);
2 //AnalRes2SW := Swap(AnalRes2);
3 Fix10Vdc := INT#4000;
4 //Fix10Vdc_1 :=WORD_TO_UINT(Analog_Res_2_Sw);
ControlVoltage
1
Auxiliaries OK
7
ControlVoltage
Auxiliaries OK
I_Start_PB
Start button
I_Stop_PB
Stop button
Stop
Stop
Slow_Inhibit
Inhibit slow button
Slow_Inhibit
Inhibit slow button
8
Main Motor start.
ControlVoltage
Auxiliaries OK
I_Stop_PB
Stop button
I_Start_PB
Start button
1
I_LowSpd_OS
Slow speed button
Slow_Inhibit
Inhibit slow button
I_Start_PB
Start button
T_StartLow
TON
In Q
T#100ms PT ET
Up
In
2
1
2
Stop
Stop
I_Motor_Thermal
Motor thermal switch
I_FanTemp_Ok
Motor (fan) thermal OK
StartMotor_Cmd
S
Motor start command

9

```
// v6m: STOP button = soft stop. StopNow replaces I_Stop_PB in rung 9.
StopNow := FALSE;
T_StopLong(In := I_Stop_PB, PT := T#1s);

IF NOT StartMotor_Cmd THEN
    SoftStop := FALSE;
ELSIF I_Stop_PB AND NOT StopPB_Mem THEN          // STOP pressed (edge)
    IF SoftStop OR NOT SoftStopEnable OR I_LowSpd_OS
      OR (Velocity_M_min <= SoftStopEndSpd) THEN
        StopNow := TRUE;                         // 2nd press / jog / slow / disabled
    ELSE
        SoftStop := TRUE;                        // start soft stop
    END_IF;
END_IF;
StopPB_Mem := I_Stop_PB;

IF T_StopLong.Q THEN
    StopNow := TRUE;                             // held 1 s = immediate
END_IF;
IF SoftStop AND I_Start_PB THEN
    SoftStop := FALSE;                           // START cancels the soft stop
END_IF;

T_SoftStop(In := SoftStop, PT := T#120s);
IF SoftStop AND ((Velocity_M_min <= SoftStopEndSpd) OR T_SoftStop.Q) THEN
    StopNow := TRUE;                             // almost stopped (or timeout)
END_IF;
```

10 Main Motor stop.



11 Hub extraction clearance.



12 Brake control.



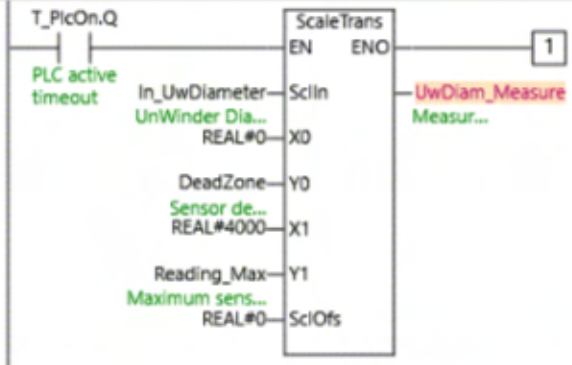
13 Cooling Fan control.



14

```
1  In_UwDiameter := INT_TO_REAL(AI_Diam_Unwinder)/REAL#1;
```

15



1

```
1  UwDiam_Act := ((UwCoreDistance - UwDiam_Measure) * REAL#2) +
   UwCoreDiameter;
2  // Con Pepperl-Fuchs UC2000-L2-U-V15:
3  // DeadZone = 60mm
4  // Reading_Max = 2000mm
5  // Con Pepperl-Fuchs OMD10M-R200-2EP-IO-V1-L:
6  // DeadZone = 200mm
7  // Reading_Max = 5000mm
```

16



17



1-8-1-1-3.Motor_Manager

0 Enable encoder reading.



1 Calculation of mm per encoder pulse.

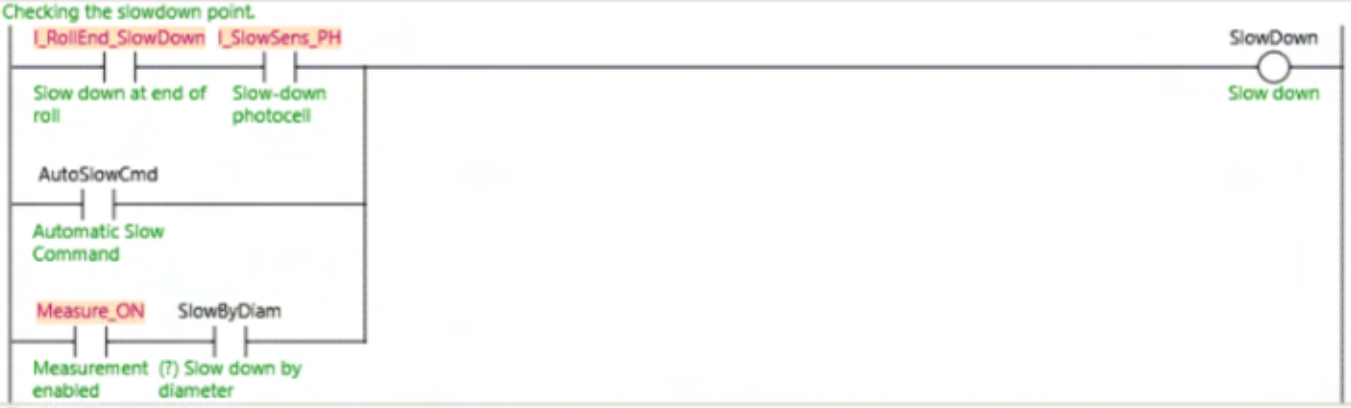
```
IF Encoder_Pulse = INT#0 THEN;
    Encoder_Pulse :=INT#1;
END_IF;
// Encoder Standard 400mm
IF  EncoderWheel_Diam = REAL#0 THEN;
    EncoderWheel_Diam := REAL#400;
END_IF;

EncoderPulse_to_mm := EncoderWheel_Diam / (INT_TO_REAL(Encoder_Pulse) *
REAL#4);
```

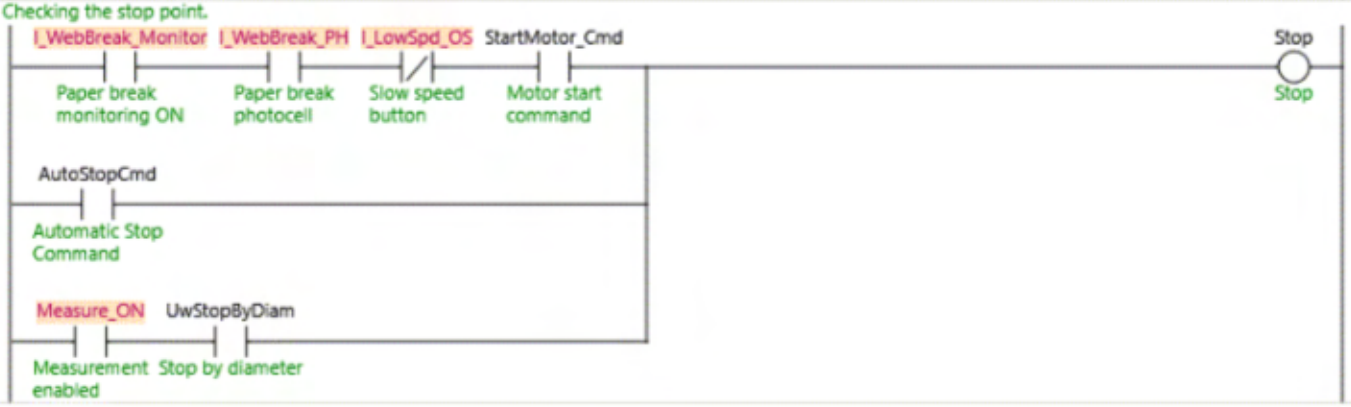
2 Reset encoder counter.



3 Checking the slowdown point.



4 Checking the stop point.



5 Check the maximum and minimum values for working speed and slow speed.

```
IF VelMax < REAL#600 THEN;
      VelMax := REAL#600;
   ELSIF VelMax > REAL#1000 THEN;
      VelMax := REAL#1000;
END_IF;
Vel_Max_mms := VelMax * REAL#1000 / REAL#60;

IF JogSpd_SP < REAL#10 THEN;
      JogSpd_SP := REAL#10;
ELSIF JogSpd_SP > REAL#70 THEN;
   JogSpd_SP := REAL#70;
END_IF;

// v6y: rewinder AUTO max speed, 0 = no limit

IF RwAutoMaxSpd > VelMax THEN
   RwAutoMaxSpd := VelMax;
ELSIF (RwAutoMaxSpd > REAL#0) AND (RwAutoMaxSpd < REAL#50) THEN
   RwAutoMaxSpd := REAL#50;
END_IF;

// taken over at standstill only

IF NOT StartMotor_Cmd THEN
   RwAutoLimAct := RwStopEnable AND (RwAutoMaxSpd > REAL#0);
   RwAutoMaxSpdAct := RwAutoMaxSpd;
END_IF;

Output_Ratio := (VelMax * Mt_To_mm) / UINT_TO_REAL(Analog_Res); //REAL#4000;
Input_Ratio := UINT_TO_REAL(Analog_Res) / VelMax ;
```

6 Reading the current value of the potentiometers. v6n: requested pot ranges, limited, applied only with the motor stopped !

```
IF WorkSpd_Max_Set < 50.0 THEN
    WorkSpd_Max_Set := 50.0;
ELSIF WorkSpd_Max_Set > VelMax THEN
    WorkSpd_Max_Set := VelMax;
END_IF;

IF FinalSpd_Max_Set < 5.0 THEN
    FinalSpd_Max_Set := 5.0;
ELSIF FinalSpd_Max_Set > WorkSpd_Max_Set THEN
    FinalSpd_Max_Set := WorkSpd_Max_Set;
END_IF;

IF NOT StartMotor_Cmd THEN
    WorkSpd_Max := WorkSpd_Max_Set;          // motor stopped: apply
    FinalSpd_Max := FinalSpd_Max_Set;
END_IF;

SpdMax_Pending := (WorkSpd_Max <> WorkSpd_Max_Set)
            OR (FinalSpd_Max <> FinalSpd_Max_Set)
            OR (RwAutoLimAct <> (RwStopEnable AND (RwAutoMaxSpd > REAL#0)))
            OR (RwAutoLimAct AND (RwAutoMaxSpdAct <> RwAutoMaxSpd));

// v6i: pots scaled to their own full scale (m/min)

IF Pot1_Work_Velocity > INT#5 THEN
    Pot1_Mem := REAL_TO_INT(INT_TO_REAL(Pot1_Work_Velocity) * WorkSpd_Max /
VelMax);
ELSE
    Pot1_Mem := INT#0;
END_IF;

IF Pot2_Slow_Velocity > INT#5 THEN
    Pot2_Mem := REAL_TO_INT(INT_TO_REAL(Pot2_Slow_Velocity) * FinalSpd_Max /
VelMax);
ELSE
    Pot2_Mem := INT#0;
END_IF;
```

7 Updating the set speed.

```
IF ManMode = FALSE THEN;

    Speed_Display := UINT_TO_REAL(Pot1_Mem) / Input_Ratio;
    LowSpd_Display := UINT_TO_REAL(Pot2_Mem) / Input_Ratio;

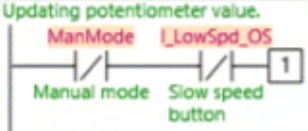
  // v6y: rewinder AUTO stop -> work speed limited

  IF RwAutoLimAct AND (Speed_Display > RwAutoMaxSpdAct) THEN
     Speed_Display := RwAutoMaxSpdAct;
  END_IF;

ELSE;
  Speed_Display := Pot1_Work_Velocity / (Analog_Res / UINT#40); // v6i: raw pot, %
  LowSpd_Display := Pot2_Slow_Velocity / (Analog_Res / UINT#40);

END_IF;
```

8 Updating potentiometer value.




```
IF SoftStop THEN
   VelSet := 0.0;                    // v6m: soft stop
ELSIF SlowDown = FALSE  THEN
   VelSet := UINT_TO_REAL(Pot1_Mem) / Input_Ratio;

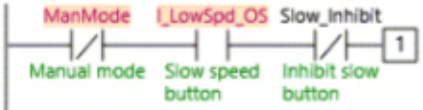
   // v6y: rewinder AUTO stop -> work speed limited

   IF RwAutoLimAct AND (VelSet > RwAutoMaxSpdAct) THEN
      VelSet := RwAutoMaxSpdAct;
   END_IF;

ELSE
   VelSet := UINT_TO_REAL(Pot2_Mem) / Input_Ratio;
END_IF;

VelSet_mms := VelSet * Mt_To_mm;
```

9 Slow speed.




```
VelSet := JogSpd_SP;
VelSet_mms := VelSet * Mt_To_mm;
```

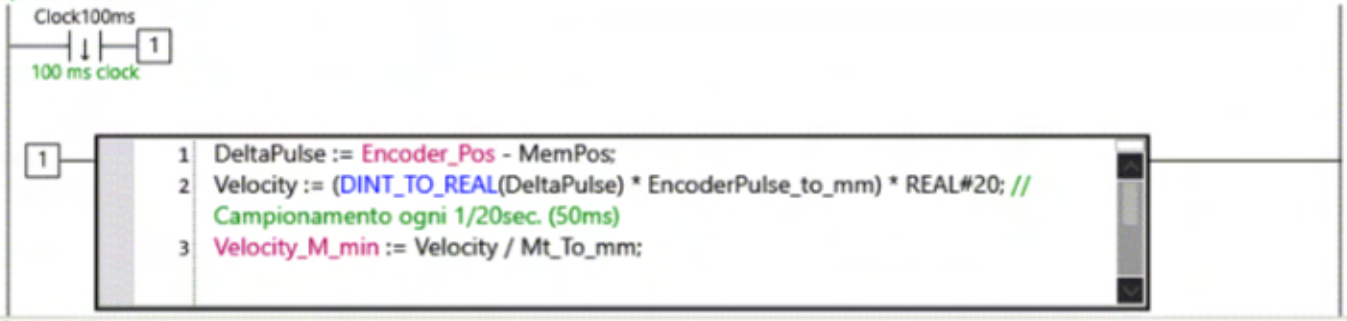
10 Check the difference between the set speed and the detected speed.

```
//SpdDelta := VelSet_mms * SpdDelta_Percent / REAL#100;
SpdDelta := SpdDelta_Percent;
SpdDeltaMax := VelSet + SpdDelta;
SpdDeltaMin := VelSet - SpdDelta;
// Too fast
IF Velocity_M_min > SpdDeltaMax THEN;
    OverSpeed := TRUE;
ELSE
    OverSpeed := FALSE;
END_IF;
// Too slow
IF Velocity_M_min < SpdDeltaMin THEN;
    UnderSpeed := TRUE;
ELSE
    UnderSpeed := FALSE;
END_IF;
```

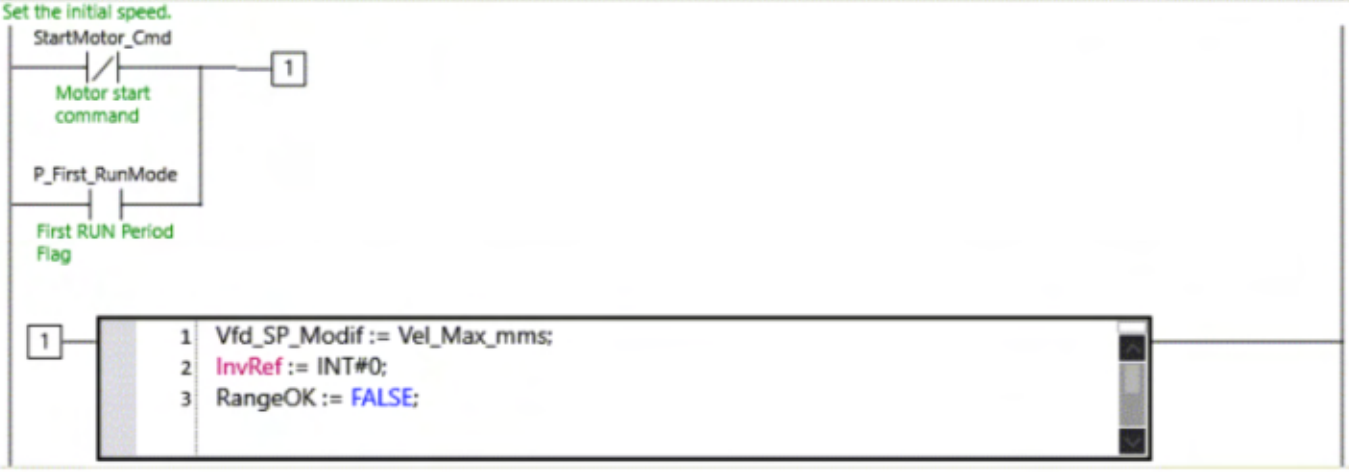
11 Read the current value of the encoder.



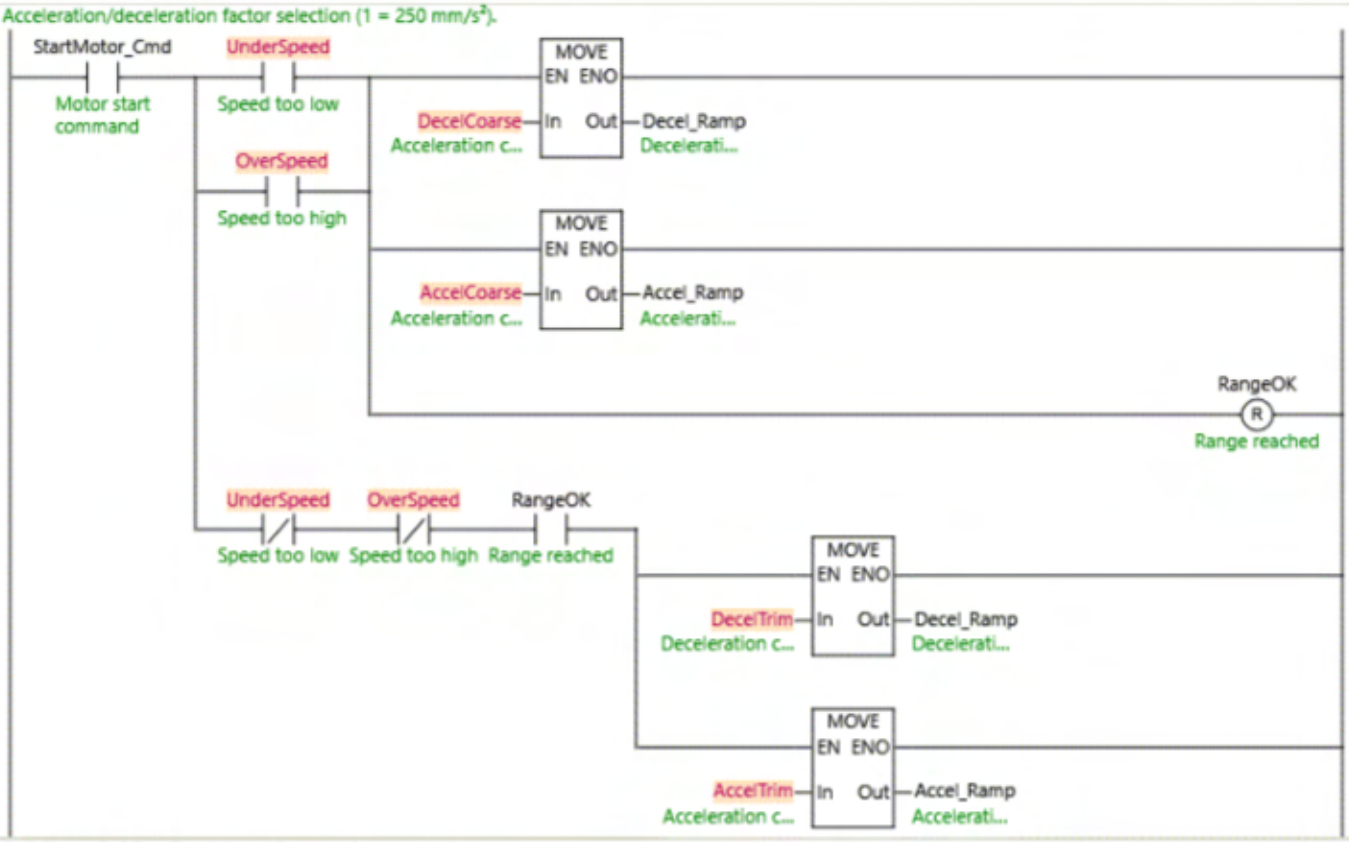
12 Speed calculation.



13 Set the initial speed.



14 Acceleration/deceleration factor selection (1 = 250 mm/s²).



15 Soft Stop - SoftStopDecel (mm/s2), same for every roll.

```
// v6r: soft stop at a REAL web decel SoftStopDecel (mm/s2), same for every roll.
// Ratio drive counts / web m/min frozen at the soft stop start (encoder = exact).

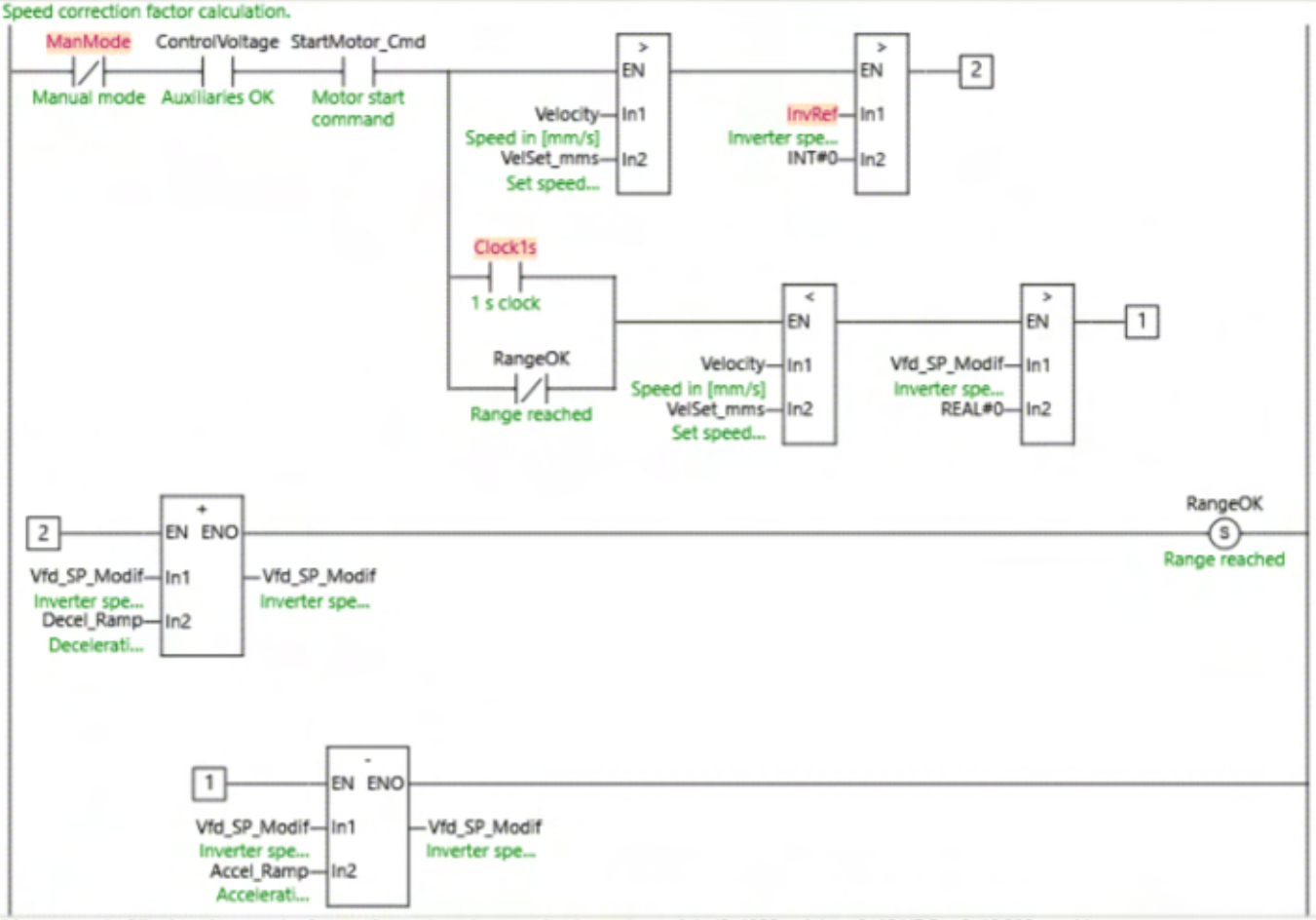
IF SoftStop AND NOT SoftStopMem THEN
    IF Velocity_M_min > 1.0 THEN
        SoftStopRatio := INT_TO_REAL(InvRef) / Velocity_M_min;
    ELSE
        SoftStopRatio := 0.0;
    END_IF;
END_IF;

SoftStopMem := SoftStop;

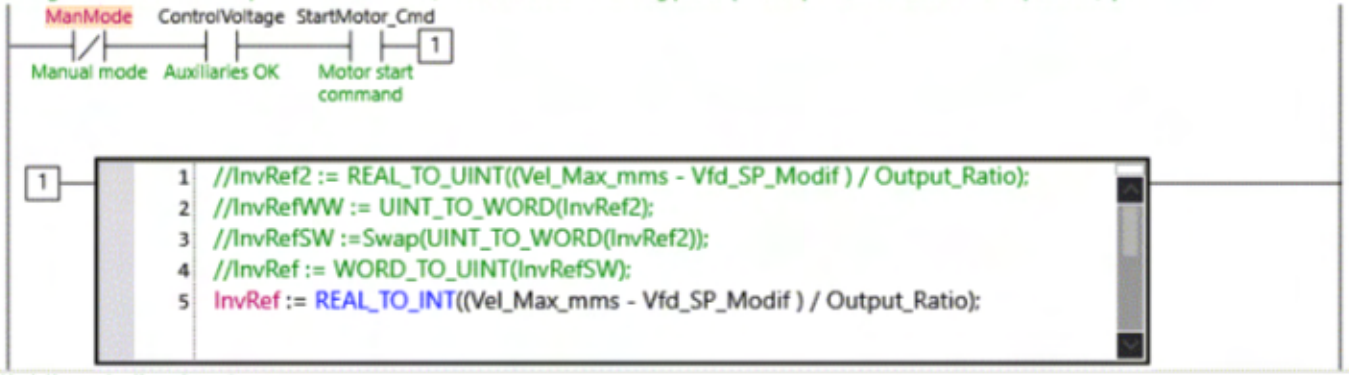
IF SoftStop AND (SoftStopDecel > 0.0) THEN
    IF SoftStopRatio > 0.0 THEN
        Decel_Ramp := SoftStopDecel * 0.06 * SoftStopRatio * Vel_Max_mms /
1000000.0;
    ELSE
        Decel_Ramp := SoftStopDecel / 250.0;          // fallback (old v6m behaviour)
    END_IF;

    IF Decel_Ramp > 8.0 THEN
        Decel_Ramp := 8.0;                    // max 9 Hz/s asked from the PLC
    END_IF;
END_IF;
```

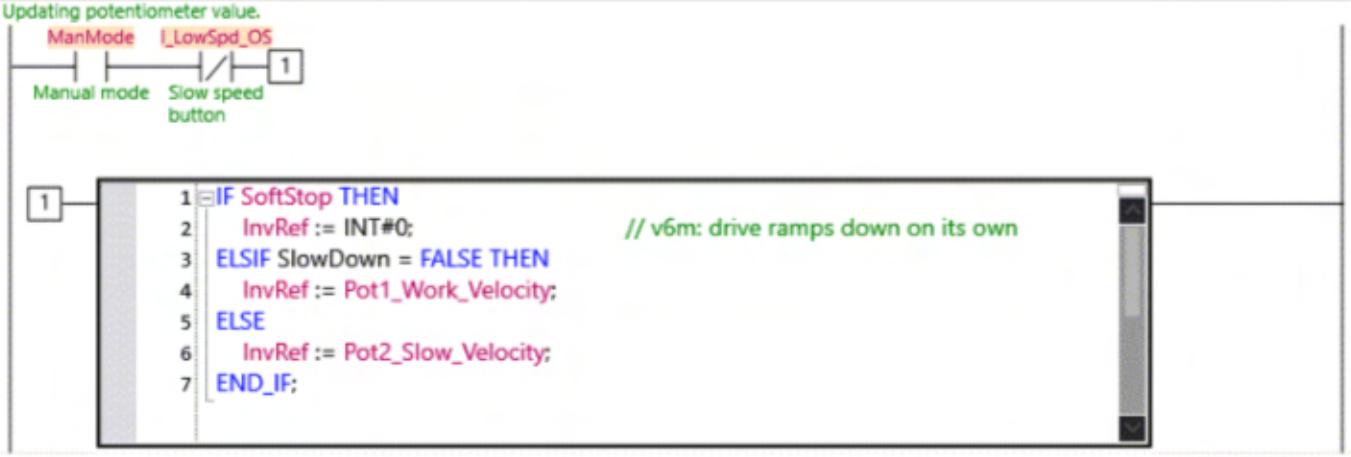
16 Speed correction factor calculation.

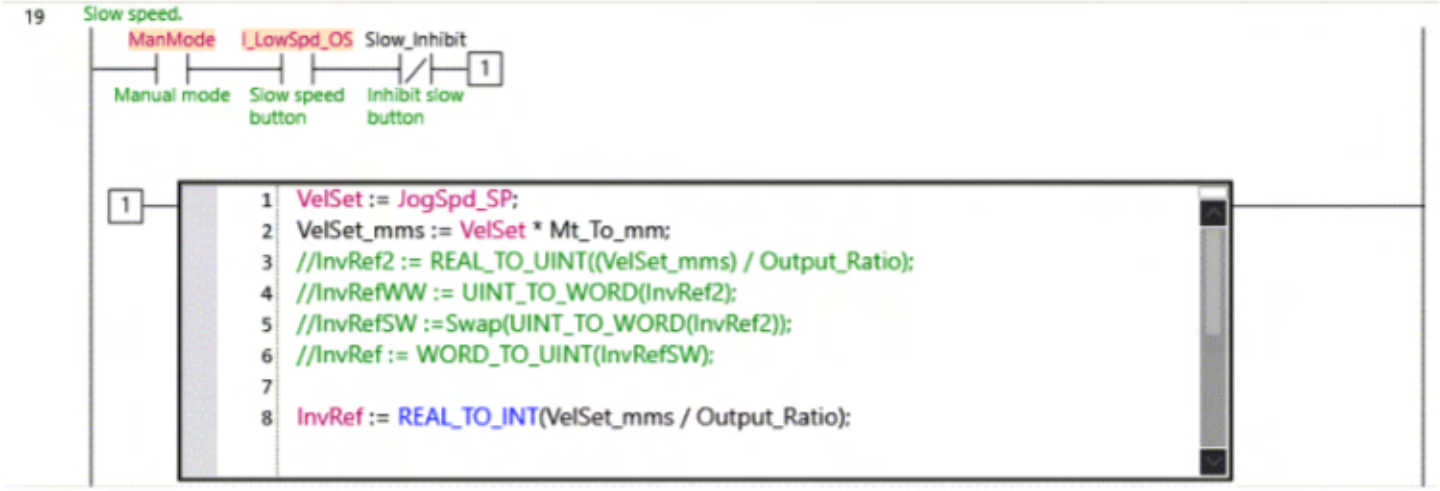


17 Management of the inverter speed reference in mm/s and conversion to analog points (0–4000 points = 0–10 VDC = 0–10,000 mm/s).

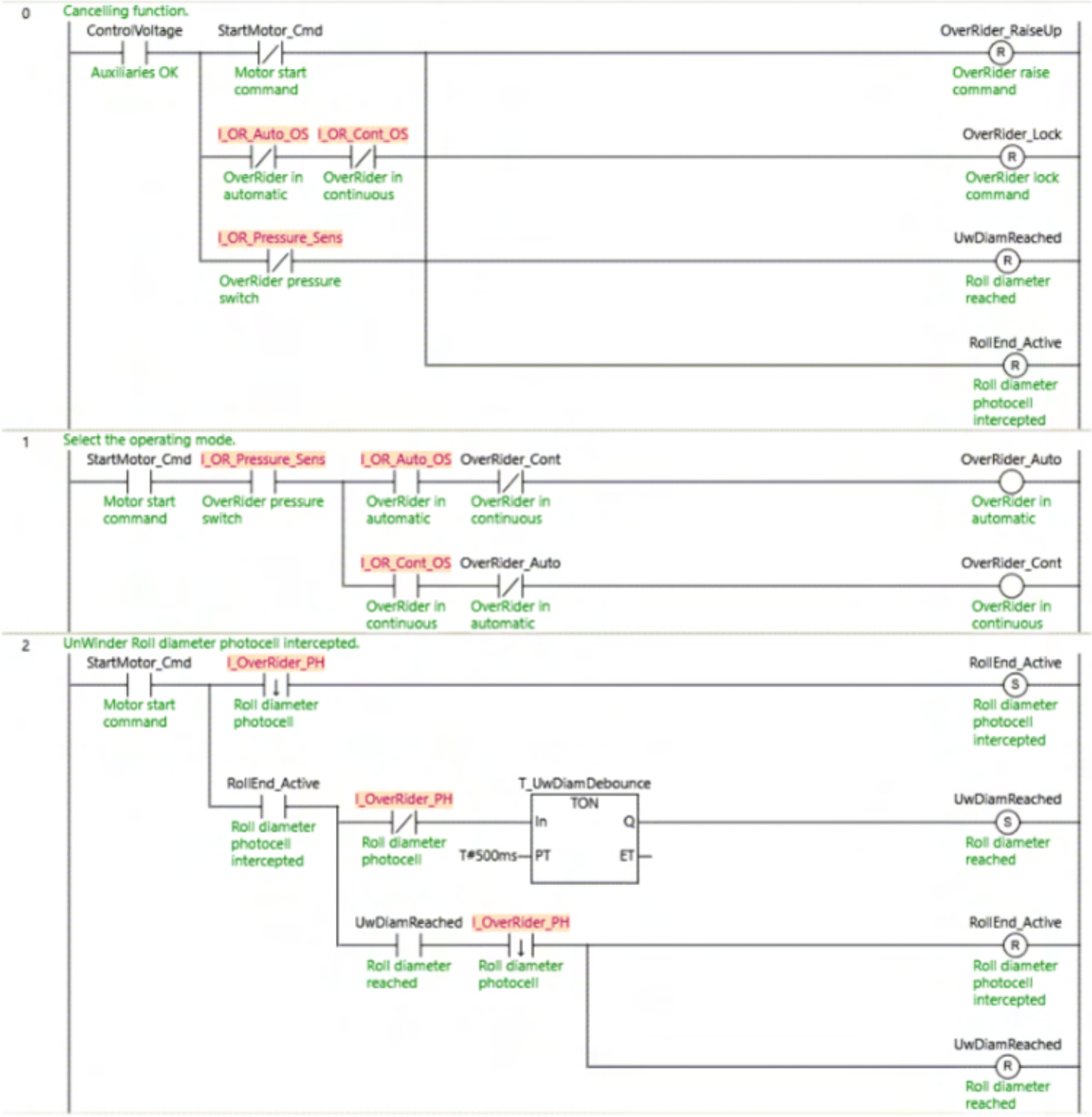


18 Updating potentiometer value.

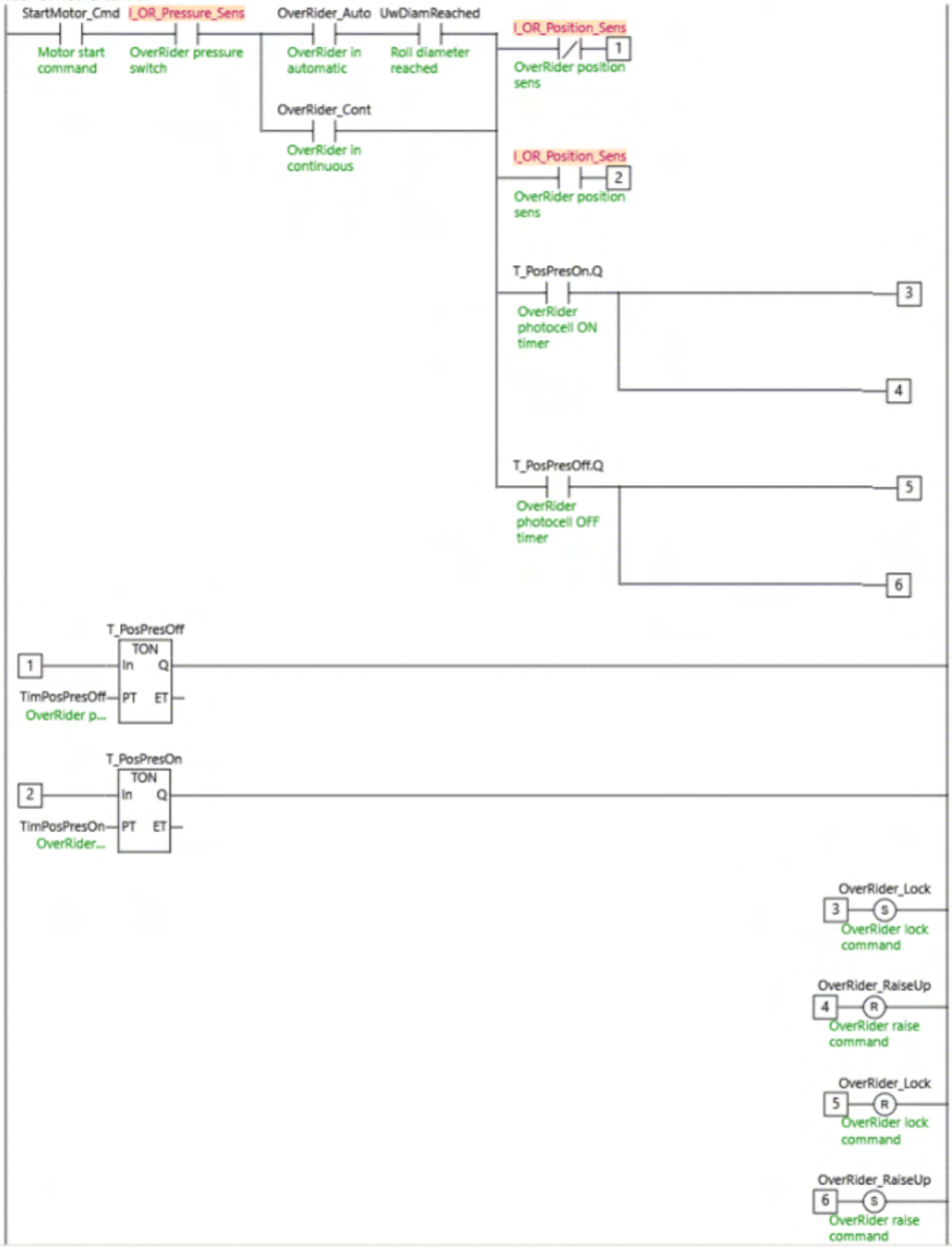



19
Slow speed.
ManMode
I_LowSpd_OS
Slow_Inhibit
1
Manual mode
Slow speed button
Inhibit slow button
1
1 VelSet := JogSpd_SP;
2 VelSet_mms := VelSet * Mt_To_mm;
3 //InvRef2 := REAL_TO_UINT((VelSet_mms) / Output_Ratio);
4 //InvRefWW := UINT_TO_WORD(InvRef2);
5 //InvRefSW :=Swap(UINT_TO_WORD(InvRef2));
6 //InvRef := WORD_TO_UINT(InvRefSW);
7
8 InvRef := REAL_TO_INT(VelSet_mms / Output_Ratio);

1-8-1-1-4.OverRider_Manager

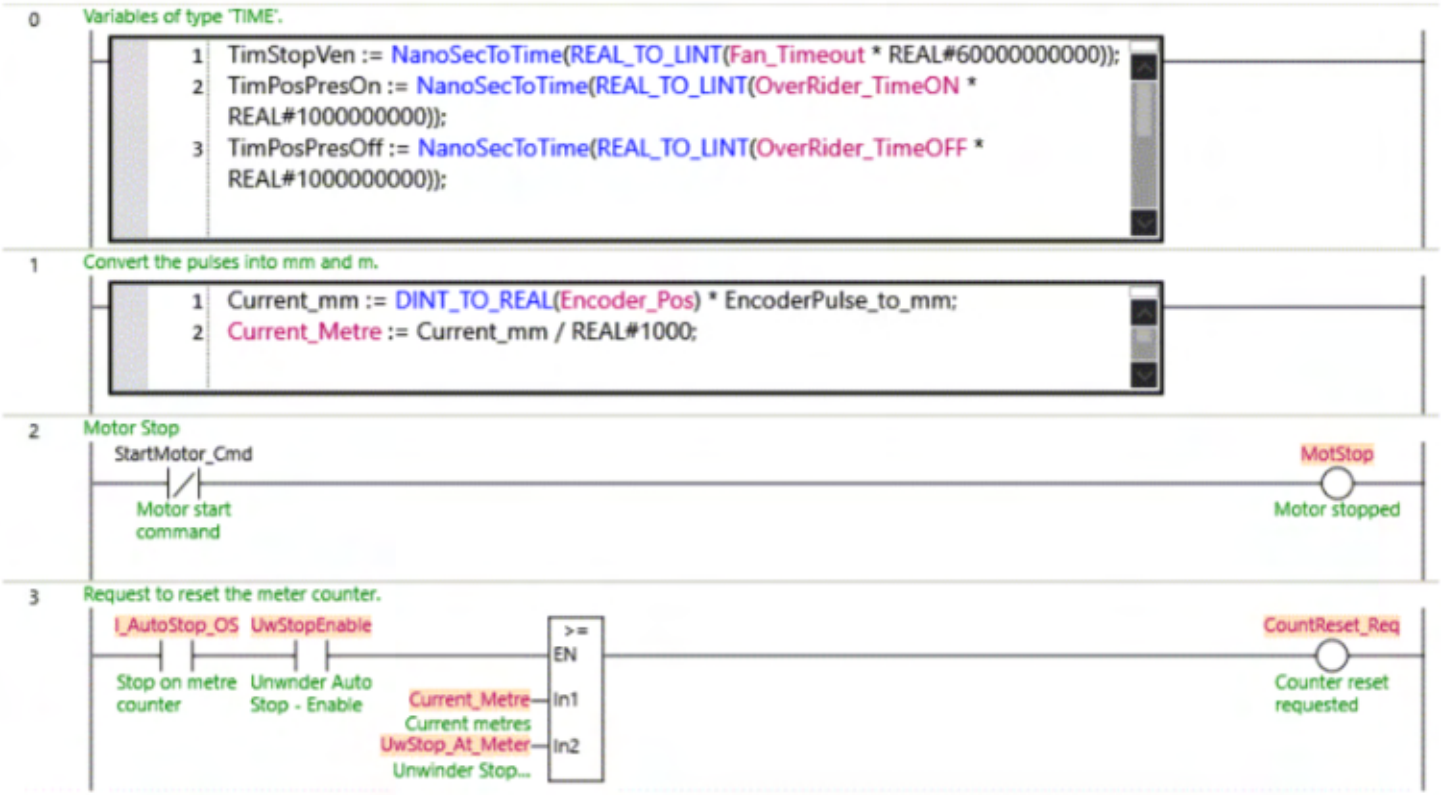


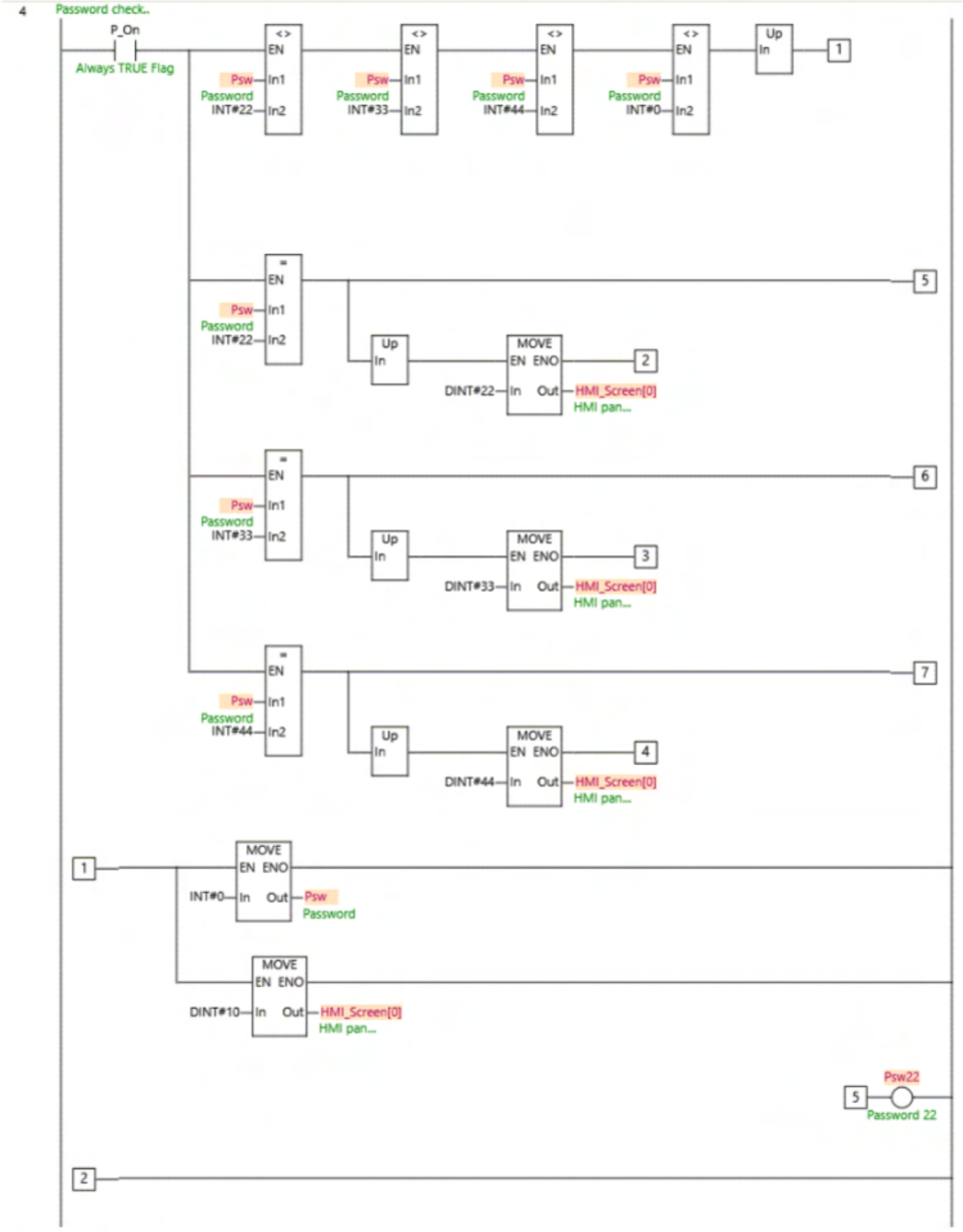
3 Rider roll Hoover control.

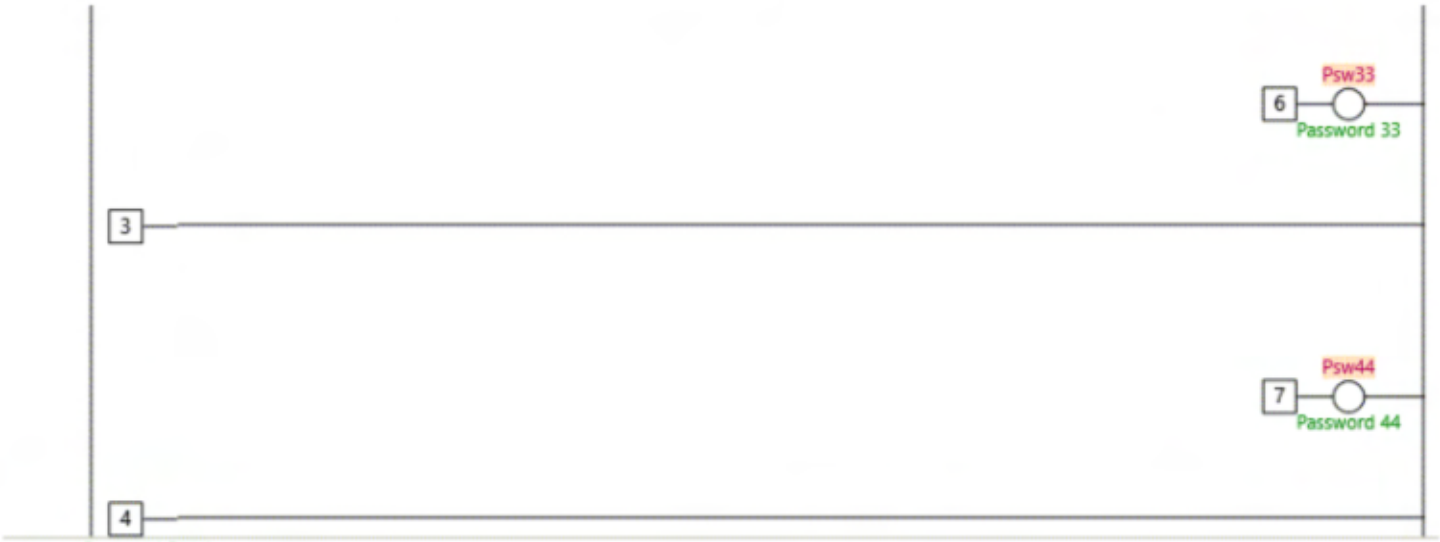



4
Rider roll raise command.
ControlVoltage
Auxiliaries OK
OverRider_RaiseUp
S
OverRider raise command
5
I_ControlVoltage
Auxiliaries ON
OverRider_RaiseUp
R
OverRider raise command

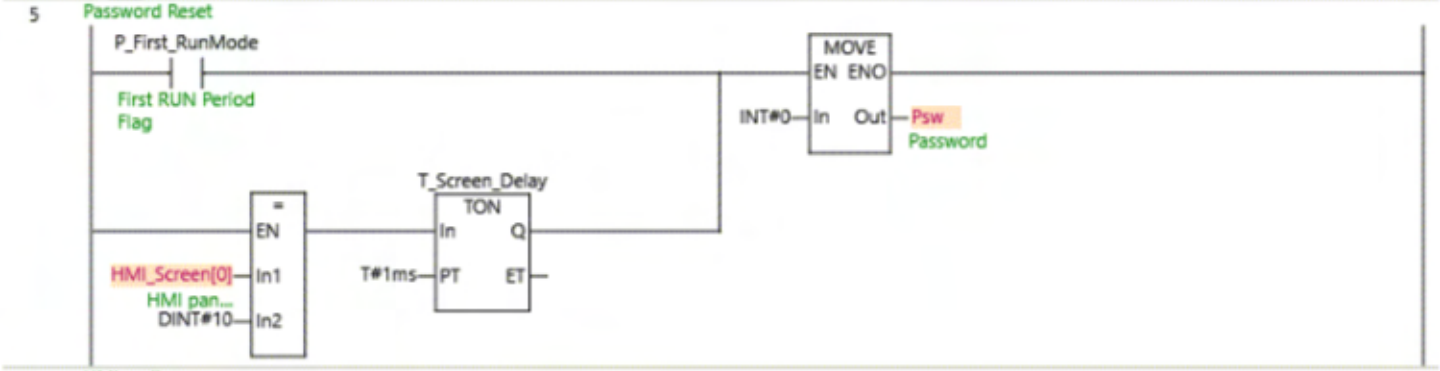
1-8-1-1-5.Console



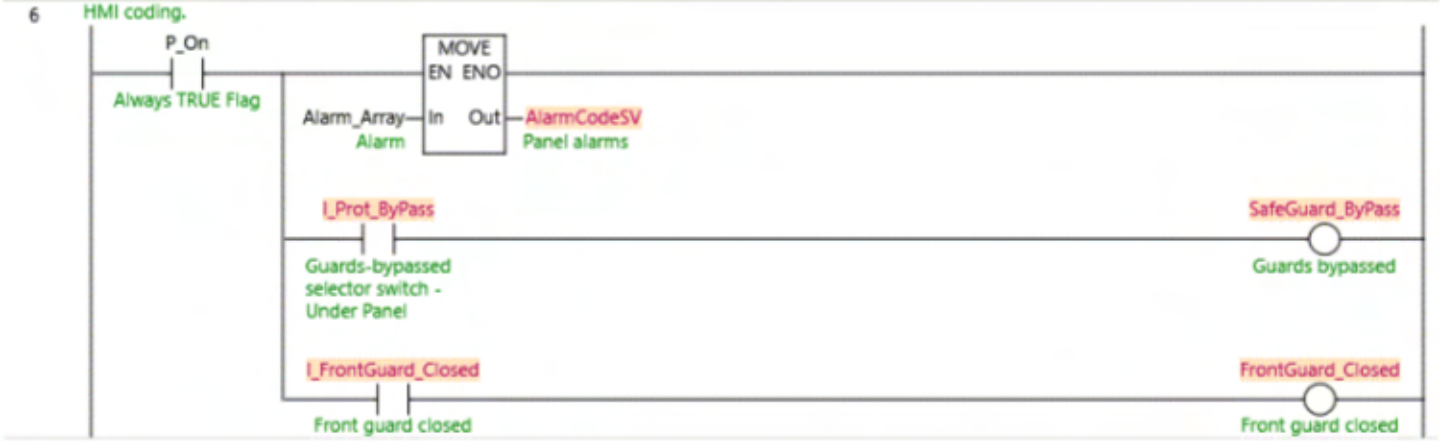

4
Password check..
P_On
Always TRUE Flag
<>
EN
In1
In2
Psw
Password
INT#22
INT#33
INT#44
INT#0
Up
In
1
=
5
6
7
2
3
4
MOVE
EN ENO
DINT#22
DINT#33
DINT#44
Out
HMI_Screen[0]
HMI pan...
DINT#10
Psw22
Password 22



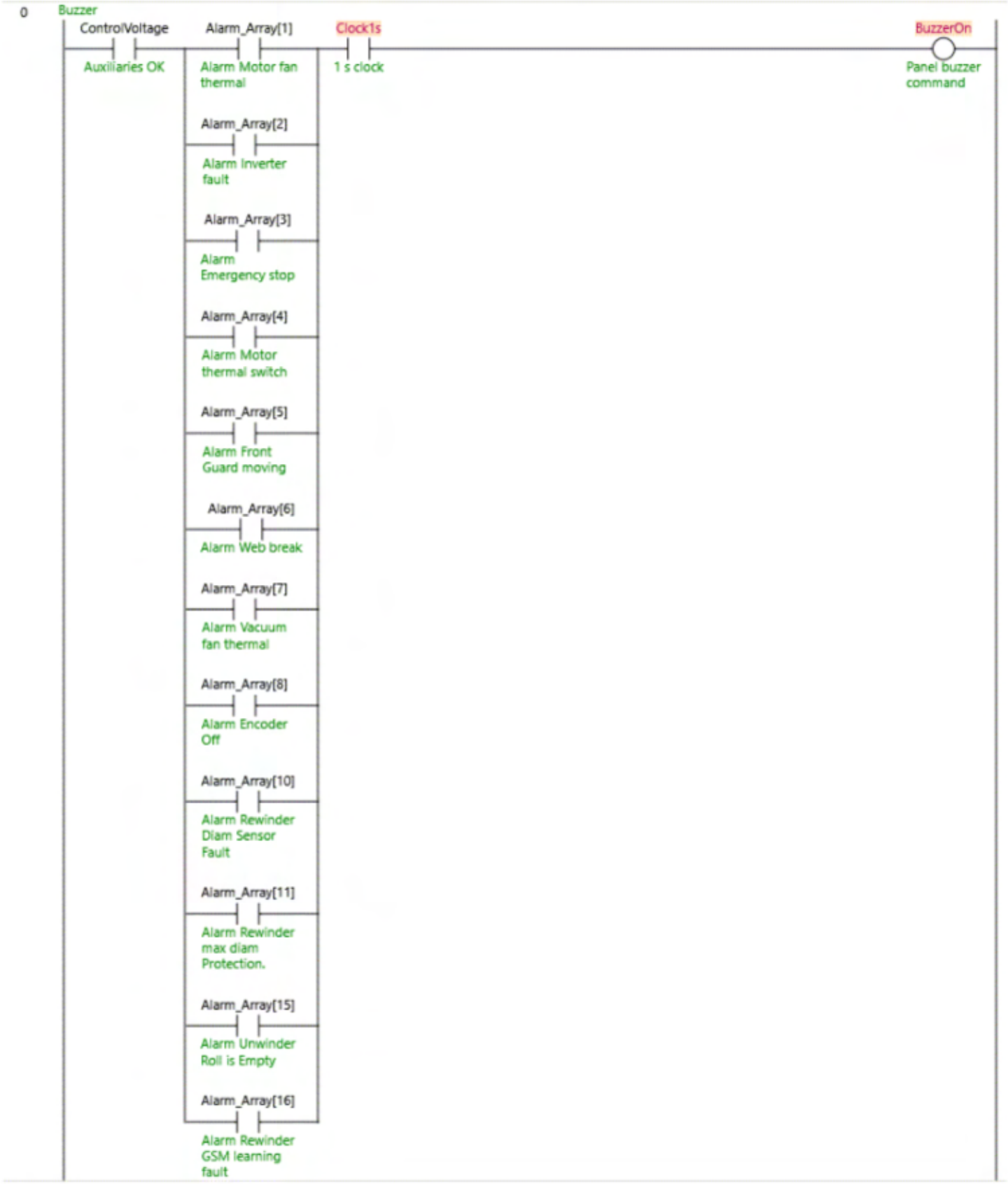
5 Password Reset



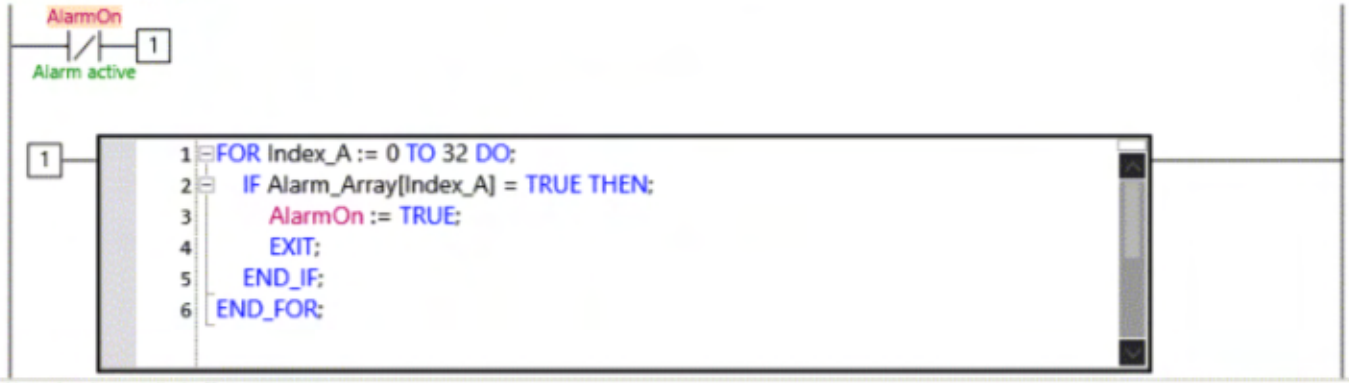
6 HMI coding.



1-8-1-1-6.Alarms



1 Check Present alarms.



2 Alarm has been called off.



3 Motor cooling Fan Alarm.



4 Inverter fault.



5 Machine in emergency mode.



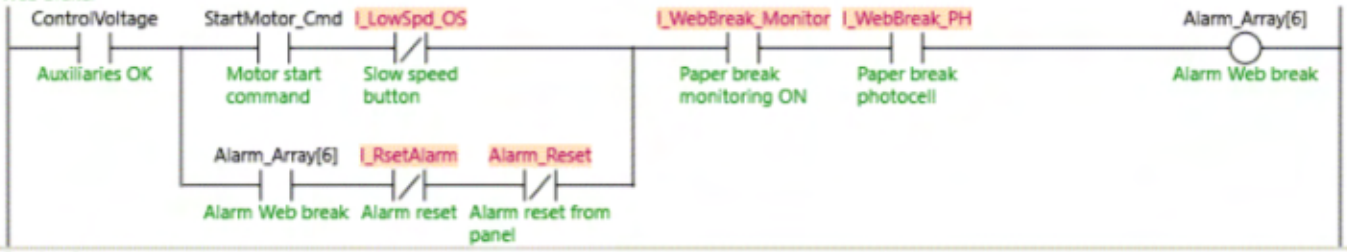
6 Motor thermal cutout.



7 Front Guard Alarm



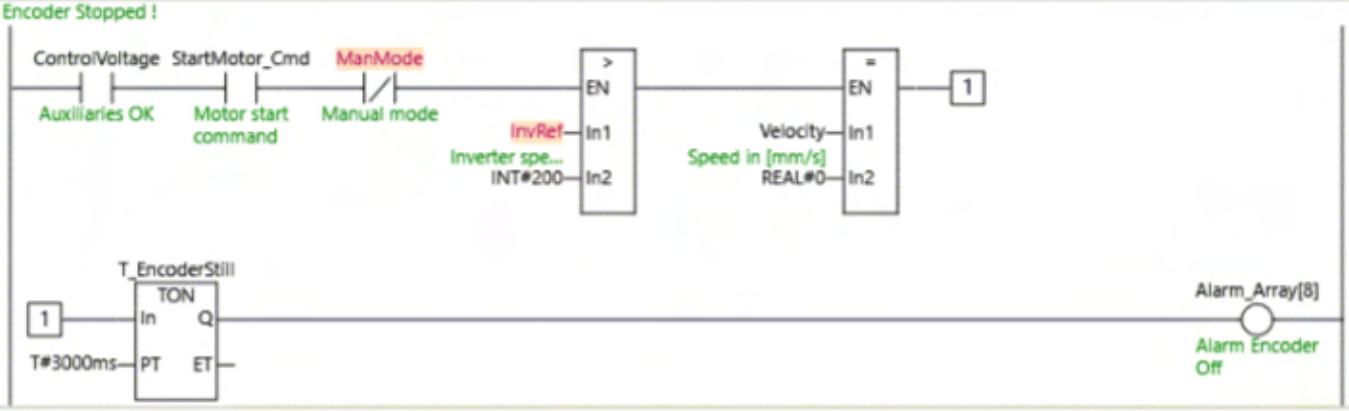
8 Web Brake.



9 Cooling fan.



10 Encoder Stopped !



11 Open-ended roll holder joint.



12 Rewinder Diameter Sensor Fault.



13 Rewinder Diameter Sensor Fault - Reset



14 Rewinder max diameter Machine protection.



15 Rewinder max diameter Machine protection - Reset



16 Rewinder GSM learning fault



17 Rewinder learning fault - Reset



18 Hub not in position.



19 Emergency Stop by Holding Stop PB 2s



20 Slow-down photocell active.



21 End-of-roll Alarm.



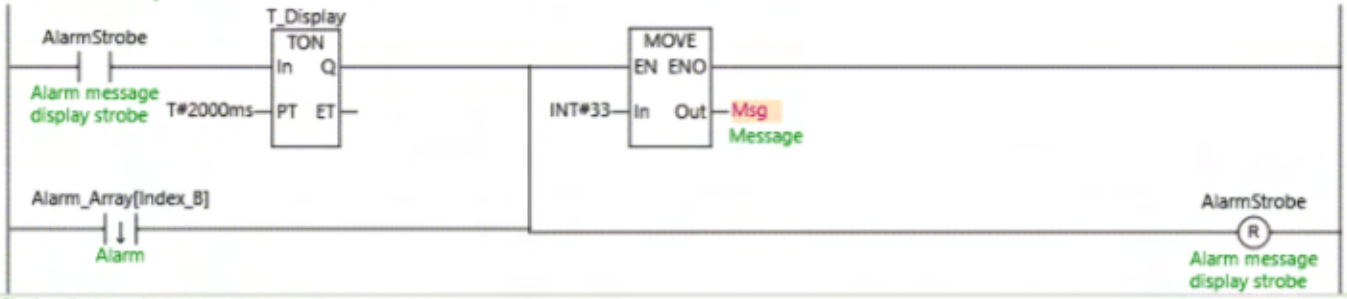
22 Unwinder Roll is Empty



23 Unwinder Roll is Empty - Reset



24 Active end-of-roll photocell.



25 Strobe displays alarm message.



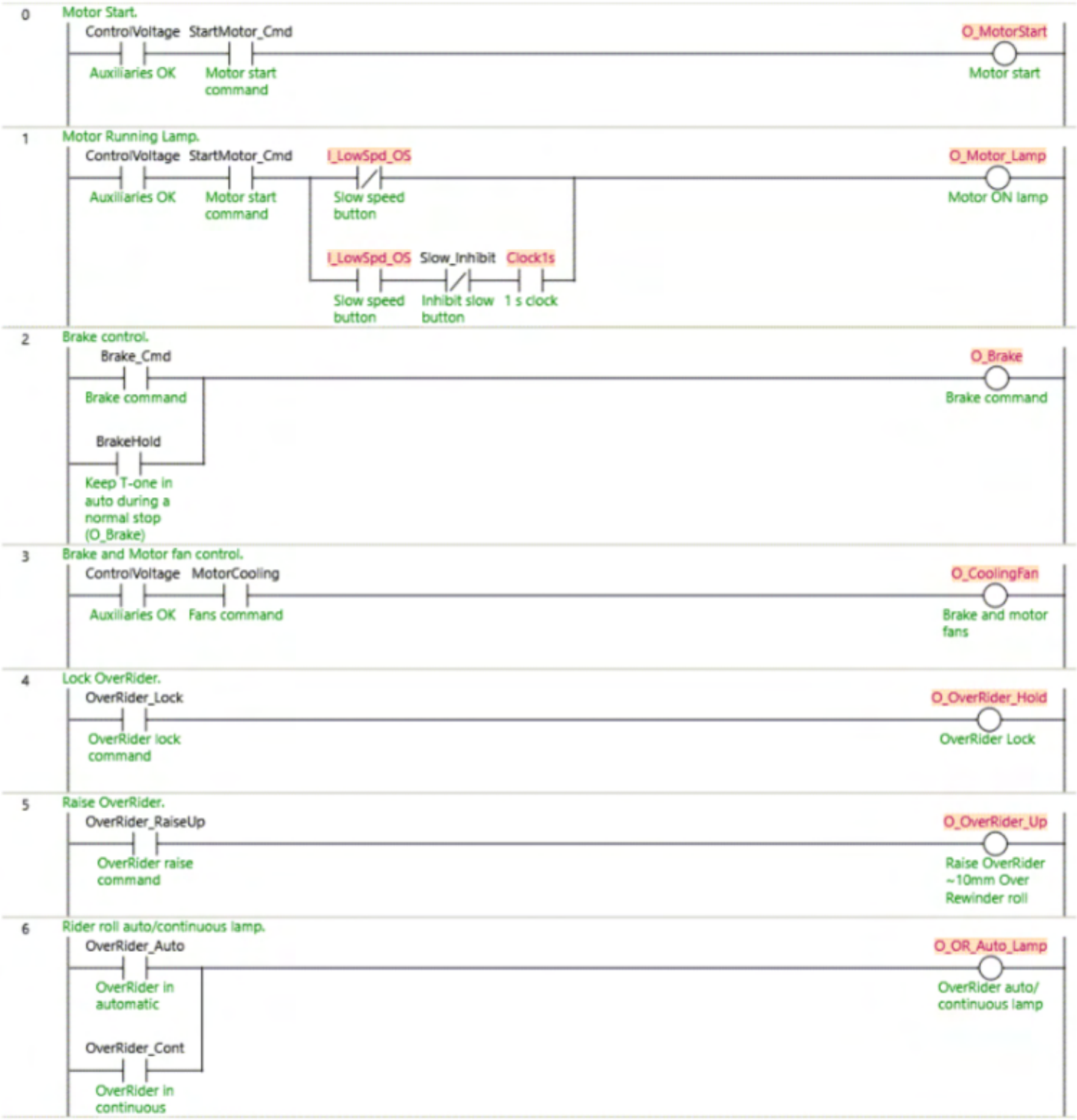
26 Alarm presence check cycle.



27 Initializing alarm message.



1-8-1-1-7.Output_Manager

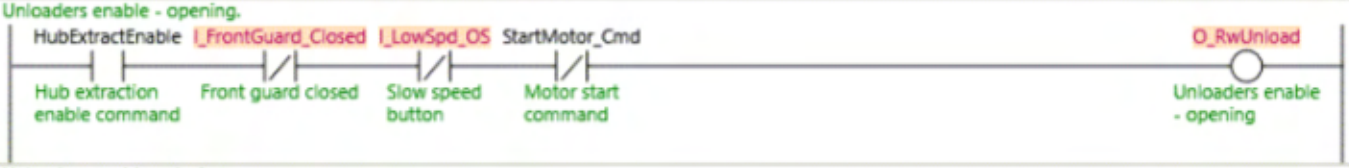



7
Alarm lamp.
ControlVoltage
Auxiliaries OK
O_Alarm_Array
Alarm lamp
Alarm_Array[1]
Alarm Motor fan thermal
Clock1s
1 s clock
Alarm_Array[2]
Alarm Inverter fault
Alarm_Array[3]
Alarm Emergency stop
Alarm_Array[4]
Alarm Motor thermal switch
Alarm_Array[5]
Alarm Front Guard moving
Alarm_Array[6]
Alarm Web break
Alarm_Array[7]
Alarm Vacuum fan thermal
Alarm_Array[8]
Alarm Encoder Off
Alarm_Array[10]
Alarm Rewinder Diam Sensor Fault
Alarm_Array[11]
Alarm Rewinder max diam Protection.
Alarm_Array[15]
Alarm Unwinder Roll is Empty
Alarm_Array[16]
Alarm Rewinder GSM learning fault

8 Guards bypassed lamp.



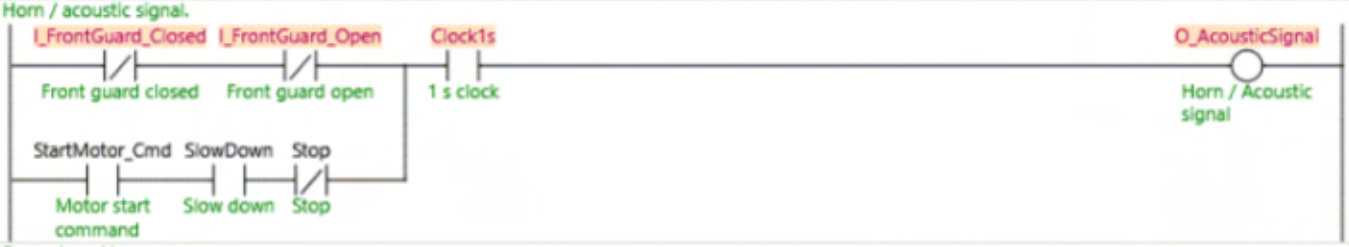
9 Unloaders enable - opening.



10 Unloaders enable - closing.



11 Horn / acoustic signal.



12 Second acc/dec ramp.



13 Front guard bypass.



1-8-1-1-8.AutoStop_Control

0 Auto Stop Process

```
(*
=============================================================
====================
  AutoStop_Control  v6y  -  Automatic slow-down / stop
  Project : 04820KA001 Mono Canada II  (NX1P2, PrimaryTask 4 ms)
  Rewinder (Rw) : SICK DT35 0-10 V -> AI_Diam_Rewinder (AD2603 Ch2)
  Unwinder (Uw) : meter counter mode + sonar diameter (UwDiam_Act)

  NAMING  Uw_ = unwinder   Rw_ = rewinder
        _mm = millimeters   _m = meters
        _Pls = encoder pulses (counts)   _Pps = pulses per second
        _Pps2 = pulses per second squared

  v6y: rewinder AUTO stop (RwStopEnable) limits the work speed to
     RwAutoMaxSpd (500 m/min), taken over at standstill only; section 6
     uses the limited speed.
  v6x: new parent roll on the unwinder -> GsmAsk (HMI popup "did you change
     the paper?"); GsmAskYes forgets the learned thickness.
  v6w: paper thickness window is LENGTH-based instead of 10 s, modulated by
     the diameter (RwWinGrow_mm 2.5 mm of growth, 4 m .. RwWinLen_m 30 m):
     same accuracy at every speed and roll size; stopped window = dropped.
     ALARM RESET also clears LearnFailCnt / StopFailCnt (alarm 16).
  v6v: BrakeHold released on the SIGNED web speed (Velocity_M_min < 5),
     so a web pulled back after the stop releases the T-one at once.
  v6u: stop learning: a stop shorter than the reaction distance or faster
     than RwDecelMax_mm is ignored (not a fault). Only a too slow stop
     (< RwDecelMin_mm) counts toward alarm 16.
  v6t: rewinder slow-down fallback while the paper thickness is not
     learned: RwSlowReq at RwTargetDiam - RwSlowFallbackMm (30 mm).
  v6s: paper change: a learning window more than RwMatChangePct (25 %) away
     from RwMat_K is ignored once, taken directly the 2nd time in a row
     (automatic). RwMatReset (HMI "NEW PAPER") forgets RwMat_K.
  v6p: section 5c: RwLearnReset clears RwStopDecelK, RwStopDecel and
     RwSlowDecelK (the v6f seed that refilled RwStopDecelK is removed).
  v6l: previous value saved before every TEACH CORE (RwAxisDist_Prev,
        UwCoreDist_Prev, Retain) + UNDO buttons (RwTeachUndo,
        UwTeachUndo): swap current <-> previous. State 4 = RESTORED.
  v6k: unwinder TEACH CORE (Core_Measure) works like the rewinder one:
        same checks, 2 s stable with sonar AVERAGE, status UwTeachState.
        Replaces General rung 15 (one immediate sonar reading).
  v6j: TEACH CORE (section 14): HMI button RwTeachCore, empty core on the
        rewinder, machine stopped -> after 2 s stable:
        RwAxisDist := RwDist + RwCoreDiameter / 2. Status in RwTeachState.
  v6i: auto: pots scaled to WorkSpd_Max / FinalSpd_Max (Motor_Manager rung 6,
        Pot1_Mem / Pot2_Mem). ManMode keeps the raw pots (percent),
        so section 6 uses the raw pots in ManMode.
  v6h: BrakeHold keeps O_Brake (T-one "reel brake automatic") ON during a
        normal stop until the web is almost stopped (< 5 m/min, max 15 s).
        Before: T-one switched to "stop" with the run command, while the
        rewinder was decelerating hard -> unwinder overran, web on the
        floor (field 10/20/2026). Used in Output_Manager rung 2.
  v6g: slow-down AND stop decel proportional to the rewinder diameter.
        The ATV930 ramp is in Hz/s (motor = rewind shaft), so the web
        decel [mm/s2] = shaft decel x radius. Field data 10/20/2026:
        24 mm/s2 at 120 mm -> 90 mm/s2 at 600 mm.
        Learned: RwSlowDecelK, RwStopDecelK [mm/s2 per mm of diameter].
```

```
55          Live:   RwSlowDecelLrn [mm/s2], RwStopDecel [pulses/s2] = K x DecelDiam.
56      v6f: learn the REAL slow-down deceleration (section 5b) -> RwSlowDecelLrn
57          (mm/s2). The ATV930 stretches its ramp with a heavy roll (no
58          braking resistor), so the PLC ramp (DecelCoarse) was too
59          optimistic: the stop came before final speed was reached.
60          Priority in section 6: RwSlowDecel (manual) > RwSlowDecelLrn
61          (learned) > DecelCoarse/DecelTrim model.
62      v6e: GSM_Learned_OK (replaces global RwLearnFault): TRUE only when
63          RwMat_K is learned, RwStopDecel is learned (if RwLearnStop) and
64          there is no learning fault. Alarm 16 uses GSM_LearnFault.
65      v6d: exact 100 ms tick (25 x 4 ms scan counter, the self-resetting
66          TON ran ~104-108 ms -> speed ~4-8 % too high)
67          standstill = <= 2 counts / 100 ms (encoder jitter)
68          stop learning skipped after jog / manual / slow-button runs and
69          when the auxiliaries drop (E-stop): RunWasJog, StopLearnValid
70          work / final speed from the potentiometers in BOTH modes
71          (Speed_Display / LowSpd_Display are not m/min in ManMode)
72          slow-down length includes the DecelTrim tail inside the
73          SpdDelta_Percent band (Motor_Manager rungs 14/15)
74      v6c: learning supervision -> RwLearnFault (alarm 16), learned paper
75          thickness RwThick_mm, plausibility limits for K and stop decel
76      v6b: RwMaxSlowAtDiam display + RwMaxSlowFallback (slow-down warning
77          while RwMat_K is not learned yet)
78      v6 : consistent internal names, units in names.
79          Globals renamed: RwRemain_m, RwStopLen_m, RwSlowLen_m,
80                      RwSlowMargin_m, UwSlowMargin_m
81      v5 : max diameter protection (always active), sensor plausibility,
82          12SA5 master (section 9), alarms in section Alarms (ladder)
83      v4b: AUTO interlock + single HMI display (SlowAtDisp / SlowAtMode)
84
    ============================================================
    ==================== *)
85
86  (* ---- 1. 100 ms tick, mm per encoder pulse ------------------------------ *)
87  (* v6d: 25 scans x 4 ms PrimaryTask = exactly 100 ms.                    *)
88  (*      Change 25 if the PrimaryTask period is changed.                  *)
89  TickCnt := TickCnt + 1;
90  IF TickCnt >= 25 THEN
91      TickCnt := 0;
92      Tick := TRUE;
93  ELSE
94      Tick := FALSE;
95  END_IF;
96
97  IF (Encoder_Pulse > 0) AND (EncoderWheel_Diam > 0.0) THEN
98      MmPerPulse := EncoderWheel_Diam / (INT_TO_REAL(Encoder_Pulse) * 4.0);
99  ELSE
100     MmPerPulse := 0.0;
101 END_IF;
102
103 (* ---- 2. Rewinder DT35 -> filtered distance -> diameter ----------------- *)
104 RwDistRaw_mm := RwDist0V + (INT_TO_REAL(AI_Diam_Rewinder) / 4000.0)
105             * (RwDist10V - RwDist0V);
106
107 IF Tick THEN
108     IF NOT RwFiltInit THEN
109         RwDist := RwDistRaw_mm;
110         RwFiltInit := TRUE;
111     ELSE
```

```
112         RwDist := RwDist + RwFilter_K * (RwDistRaw_mm - RwDist);
113     END_IF;
114 END_IF;
115
116 RwSensFault := (AI_Diam_Rewinder < RwRawMin) OR (AI_Diam_Rewinder >
    RwRawMax)
117             OR (RwDist > RwAxisDist);
118
119 RwDiam := 2.0 * (RwAxisDist - RwDist);
120 IF RwDiam < 0.0 THEN
121     RwDiam := 0.0;
122 END_IF;
123 (* RwAxisDist = DT35 reference to rewind shaft centre (mm), set manually *)
124
125 (* v6g: diameter used by the decel model (last valid on sensor fault)      *)
126 IF NOT RwSensFault THEN
127     DecelDiam := RwDiam;
128 ELSE
129     DecelDiam := RwLastValidDiam;
130 END_IF;
131 IF DecelDiam < 100.0 THEN
132     DecelDiam := 100.0;                    (* core *)
133 END_IF;
134
135 (* ---- 3. Actual speed from encoder, 100 ms window ---------------------- *)
136 IF Tick THEN
137     IF NOT SpdInit THEN
138         EncPrev := Encoder_Pos;
139         SpdInit := TRUE;
140     END_IF;
141     SpdAct_Pps := DINT_TO_REAL(ABS(Encoder_Pos - EncPrev)) * 10.0;
142     EncPrev := Encoder_Pos;
143 END_IF;
144 RwSpeed := SpdAct_Pps * MmPerPulse * 0.06;         (* m/min *)
145
146 (* ---- 4a. v6s: NEW PAPER button: forget the learned thickness ---------- *)
147 (* RwMat_K = 0 -> the first valid 10 s window sets it directly.           *)
148 (* HMI momentary button: must be HELD 3 s, then the reset is done once and *)
149 (* the PLC clears the bit (HMI lamp goes off = done).                     *)
150 T_MatReset(In := RwMatReset, PT := T#3s);
151 IF T_MatReset.Q THEN
152     RwMat_K := 0.0;
153     LearnFailCnt := 0;
154     MatChangeCnt := 0;
155     RwMatReset := FALSE;
156 END_IF;
157
158 (* ---- 4. Learn material factor  K = dDiam/dPulse * Diam ----------------- *)
159 IF Tick THEN
160     LearnWinTicks := LearnWinTicks + 1;
161     IF NOT LearnInit THEN
162         LearnWinStartPos := Encoder_Pos;  LearnWinStartDiam := RwDiam;
163         LearnWinTicks := 0;
164         LearnWinRunning := O_MotorStart;
165         LearnInit := TRUE;
166     ELSE
167         LearnWinRunning := LearnWinRunning AND O_MotorStart;
168         (* v6w: LENGTH-based window, modulated by the diameter: closes    *)
169         (* when the roll should have grown RwWinGrow_mm (2.5 mm), so the  *)
```

```
170     (* accuracy vs DT35 noise is the same at every speed and size.    *)
171     (* Pulses = growth x diameter / K (K = learned RwMat_K).          *)
172     (* Limits: min 4 m, max RwWinLen_m (30 m). K not learned: max.    *)
173     (* Motor stopped during the window -> window dropped (no fault).  *)
174     IF MmPerPulse > 0.0 THEN
175       IF (RwMat_K > 0.0) AND (RwWinGrow_mm > 0.0) THEN
176         LearnWinLen_Pls := RwWinGrow_mm * MAX(RwDiam, REAL#100.0) /
    RwMat_K;
177       ELSE
178         LearnWinLen_Pls := RwWinLen_m * 1000.0 / MmPerPulse;
179       END_IF;
180       IF LearnWinLen_Pls < 4000.0 / MmPerPulse THEN
181         LearnWinLen_Pls := 4000.0 / MmPerPulse;            (* 4 m *)
182       END_IF;
183       IF (RwWinLen_m > 0.0)
184         AND (LearnWinLen_Pls > RwWinLen_m * 1000.0 / MmPerPulse) THEN
185         LearnWinLen_Pls := RwWinLen_m * 1000.0 / MmPerPulse;
186       END_IF;
187     ELSE
188       LearnWinLen_Pls := 300000.0;
189     END_IF;
190     IF (DINT_TO_REAL(ABS(Encoder_Pos - LearnWinStartPos)) >= LearnWinLen_Pls)
191       OR NOT LearnWinRunning THEN
192       LearnWin_Pls := ABS(Encoder_Pos - LearnWinStartPos);
193       LearnWinDiamDelta := RwDiam - LearnWinStartDiam;
194       (* A window SHOULD learn: ran all the time, enough length.  *)
195       (* DT35 OK. Then it must give a plausible paper thickness. *)
196       IF LearnWinRunning AND (LearnWin_Pls >= RwMinPulses)
197         AND NOT RwSensFault THEN
198         LearnWinOk := FALSE;
199         IF (LearnWinDiamDelta > 0.0) AND (MmPerPulse > 0.0) THEN
200           MatK_Window := (LearnWinDiamDelta / DINT_TO_REAL(LearnWin_Pls))
201                 * ((RwDiam + LearnWinStartDiam) / 2.0);
202           (* K = 2 * thickness * mm-per-pulse / PI          *)
203           ThickWin_mm := MatK_Window * 3.14159 / (2.0 * MmPerPulse);
204           IF (ThickWin_mm >= RwThickMin_mm)
205             AND (ThickWin_mm <= RwThickMax_mm) THEN
206             LearnWinOk := TRUE;
207             IF RwMat_K <= 0.0 THEN
208               RwMat_K := MatK_Window;
209               MatChangeCnt := 0;
210             ELSIF (RwMatChangePct > 0.0)
211                AND (ABS(MatK_Window - RwMat_K)
212                   > RwMat_K * RwMatChangePct / 100.0) THEN
213               (* v6s: far from the learned value: 1 window = ignored *)
214               (* (outlier), 2 in a row = paper changed -> take it.  *)
215               MatChangeCnt := MatChangeCnt + 1;
216               IF MatChangeCnt >= 2 THEN
217                 RwMat_K := MatK_Window;
218                 MatChangeCnt := 0;
219               END_IF;
220             ELSE
221               MatChangeCnt := 0;
222               RwMat_K := RwMat_K + RwLearn_K * (MatK_Window - RwMat_K);
223             END_IF;
224           END_IF;
225         END_IF;
226         IF LearnWinOk THEN
227           LearnFailCnt := 0;
```

```
            ELSIF LearnFailCnt < 1000 THEN
               LearnFailCnt := LearnFailCnt + 1;  (* no growth / implausible *)
            END_IF;
         END_IF;
         LearnWinStartPos := Encoder_Pos;  LearnWinStartDiam := RwDiam;
         LearnWinTicks := 0;
         LearnWinRunning := O_MotorStart;
      END_IF;
   END_IF;
END_IF;

(* ---- 5. Learn real stop deceleration on every stop ---------------------- *)
(* v6d: a run with jog / manual / slow button uses the 2nd ramp (O_AccDec2) *)
(*      -> its stop must not be learned. Latched for the whole run because  *)
(*      I_LowSpd_OS is already released on the stop edge.                   *)
IF O_MotorStart AND NOT MotorRunMem THEN          (* start edge *)
   RunWasJog := FALSE;
END_IF;
IF O_MotorStart AND (ManMode OR I_LowSpd_OS) THEN
   RunWasJog := TRUE;
END_IF;

IF MotorRunMem AND NOT O_MotorStart THEN          (* stop command edge *)
   StopMeasActive := TRUE;
   StopStartPos := Encoder_Pos;
   StopStartSpd_Pps := SpdAct_Pps;
   StopStartDiam := DecelDiam;
   StandstillTicks := 0;
   (* STOP button and automatic stop take the same path (drive DI1 and    *)
   (* O_Brake -> T-one drop together, electrical plan sheets 1/9/15),     *)
   (* so both are learned.                                                *)
   StopLearnValid := ControlVoltage AND NOT RunWasJog AND NOT ManMode;
END_IF;
MotorRunMem := O_MotorStart;

IF O_MotorStart THEN
   StopMeasActive := FALSE;                  (* restarted: abort *)
ELSIF StopMeasActive AND Tick THEN
   IF NOT ControlVoltage THEN
      StopLearnValid := FALSE;               (* E-stop during the stop *)
   END_IF;
   IF SpdAct_Pps <= 20.0 THEN                (* <= 2 counts / 100 ms *)
      StandstillTicks := StandstillTicks + 1;
   ELSE
      StandstillTicks := 0;
   END_IF;
   IF StandstillTicks >= 3 THEN              (* standstill 300 ms *)
      StopMeasActive := FALSE;
      StopMeas_Pls := DINT_TO_REAL(ABS(Encoder_Pos - StopStartPos));
      IF RwLearnStop AND StopLearnValid
        AND (StopStartSpd_Pps * MmPerPulse * 0.06 >= RwMinLearnSpd) THEN
         (* This stop SHOULD learn: result must be plausible       *)
         (* v6u: a stop shorter than the reaction distance, or     *)
         (* faster than RwDecelMax_mm, is NOT a fault: the web     *)
         (* stopped within a few mm, nothing to learn (trace       *)
         (* 10/28/2026: 23 m/min -> stopped in 102 pulses = 10 mm). *)
         (* Only a too SLOW stop (< RwDecelMin_mm) counts as fail.  *)
         StopLearnOk := FALSE;
         IF StopMeas_Pls > StopStartSpd_Pps * RwTreact THEN
```

```
287             StopDecelMeas := (StopStartSpd_Pps * StopStartSpd_Pps)
288                     / (2.0 * (StopMeas_Pls - StopStartSpd_Pps * RwTreact));
289             StopDecelMeas_mm := StopDecelMeas * MmPerPulse;  (* mm/s2 *)
290             IF (StopDecelMeas_mm >= RwDecelMin_mm)
291               AND (StopDecelMeas_mm <= RwDecelMax_mm) THEN
292                StopLearnOk := TRUE;
293                (* v6g: learn per mm of diameter *)
294                DecelK_Meas := StopDecelMeas_mm / ((StopStartDiam + DecelDiam) /
   2.0);
295                IF RwStopDecelK <= 0.0 THEN
296                   RwStopDecelK := DecelK_Meas;
297                ELSE
298                   RwStopDecelK := RwStopDecelK + 0.3 * (DecelK_Meas -
   RwStopDecelK);
299                END_IF;
300                StopFailCnt := 0;
301             ELSIF StopDecelMeas_mm < RwDecelMin_mm THEN
302                IF StopFailCnt < 1000 THEN
303                   StopFailCnt := StopFailCnt + 1;    (* stop too long *)
304                END_IF;
305             END_IF;                          (* too fast: ignored *)
306          END_IF;                             (* too short: ignored *)
307       END_IF;
308    END_IF;
309 END_IF;
310
311 (* ---- 5b. v6f: learn real slow-down deceleration ----------------------- *)
312 (* From the SlowDown edge (at speed) until final speed is reached, or     *)
313 (* until the stop comes first. a = (v0^2 - v1^2) / (2 * distance).        *)
314 (* Not learned in jog / manual / slow button (2nd ramp).                  *)
315 IF SlowDown AND NOT SlowMem AND O_MotorStart
316    AND NOT ManMode AND NOT I_LowSpd_OS
317    AND (SpdAct_Pps > SpdFinal_Pps + SpdBand_Pps) THEN
318     SlowMeasActive := TRUE;
319     SlowStartPos := Encoder_Pos;
320     SlowStartSpd_Pps := SpdAct_Pps;
321     SlowStartDiam := DecelDiam;
322 END_IF;
323 SlowMem := SlowDown;
324
325 IF SlowMeasActive THEN
326    IF NOT SlowDown OR ManMode OR I_LowSpd_OS THEN
327       SlowMeasActive := FALSE;                (* aborted *)
328    ELSIF Tick AND ((SpdAct_Pps <= SpdFinal_Pps + 0.25 * SpdBand_Pps)
329                OR NOT O_MotorStart) THEN      (* final reached or stopped *)
330       SlowMeasActive := FALSE;
331       SlowMeas_Pls := DINT_TO_REAL(ABS(Encoder_Pos - SlowStartPos));
332       IF (SlowMeas_Pls > 0.0) AND (MmPerPulse > 0.0)
333         AND ((SlowStartSpd_Pps - SpdAct_Pps) * MmPerPulse * 0.06 >= 10.0) THEN
334          SlowDecelMeas_mm := (SlowStartSpd_Pps * SlowStartSpd_Pps
335                          - SpdAct_Pps * SpdAct_Pps)
336                         / (2.0 * SlowMeas_Pls) * MmPerPulse;  (* mm/s2 *)
337          IF (SlowDecelMeas_mm >= RwDecelMin_mm)
338            AND (SlowDecelMeas_mm <= RwDecelMax_mm) THEN
339             (* v6g: learn per mm of diameter *)
340             DecelK_Meas := SlowDecelMeas_mm / ((SlowStartDiam + DecelDiam) /
   2.0);
341             IF RwSlowDecelK <= 0.0 THEN
342                RwSlowDecelK := DecelK_Meas;
```

```
343                 ELSE
344                     RwSlowDecelK := RwSlowDecelK + 0.5 * (DecelK_Meas -
    RwSlowDecelK);
345                 END_IF;
346             END_IF;
347         END_IF;
348     END_IF;
349 END_IF;
350
351 (* ---- 5c. v6g: live decels = learned K x current diameter ---------- *)
352 (* v6p: the one-time v6f seed is removed (it re-filled RwStopDecelK from    *)
353 (* RwStopDecel, so the learning could never be reset).                     *)
354 (* RwLearnReset (HMI / Watch tab, PLC resets it): clear the learned decels. *)
355 (* Both are re-learned at the next auto slow-down / stop >= RwMinLearnSpd.  *)
356 IF RwLearnReset THEN
357     RwStopDecelK := 0.0;
358     RwStopDecel := 0.0;
359     RwSlowDecelK := 0.0;
360     StopFailCnt := 0;
361     RwLearnReset := FALSE;
362 END_IF;
363 IF (RwStopDecelK > 0.0) AND (MmPerPulse > 0.0) THEN
364     RwStopDecel := RwStopDecelK * DecelDiam / MmPerPulse;     (* pulses/s2 *)
365 END_IF;
366 IF RwSlowDecelK > 0.0 THEN
367     RwSlowDecelLrn := RwSlowDecelK * DecelDiam;               (* mm/s2 *)
368 ELSE
369     RwSlowDecelLrn := 0.0;
370 END_IF;
371
372 (* ---- 6. Lengths in pulses: stop, slow-down, remaining ------------------ *)
373 (* Stop length uses the ACTUAL speed (precision)                           *)
374 IF RwStopDecel > 0.0 THEN
375     StopLen_Pls := SpdAct_Pps * RwTreact
376                 + (SpdAct_Pps * SpdAct_Pps) / (2.0 * RwStopDecel);
377 ELSE
378     StopLen_Pls := SpdAct_Pps * (RwTreact + RwTstopDef / 2.0);
379 END_IF;
380 StopLen_Pls := StopLen_Pls * RwTrim;
381
382 (* Slow-down uses SpdRef = MAX(actual, work-speed setpoint):               *)
383 (* stable value, shown to the operator even before start                   *)
384 SlowLenBase_Pls := 0.0;  RwSlowLen_Pls := 0.0;  UwSlowLen_Pls := 0.0;
385 IF MmPerPulse > 0.0 THEN
386     (* v6f: manual > learned > PLC ramp model *)
387     IF RwSlowDecel > 0.0 THEN
388         SlowDecel_Pps2 := RwSlowDecel / MmPerPulse;
389         SlowOneRamp := TRUE;
390     ELSIF RwSlowDecelLrn > 0.0 THEN
391         SlowDecel_Pps2 := RwSlowDecelLrn / MmPerPulse;  (* real, incl. trim tail *)
392         SlowOneRamp := TRUE;
393     ELSE
394         SlowDecel_Pps2 := DecelCoarse * 250.0 / MmPerPulse; (* 1 = 250 mm/s2 *)
395         SlowOneRamp := FALSE;
396     END_IF;
397     (* v6d: DecelTrim ramp used inside the +/- SpdDelta_Percent band *)
398     IF DecelTrim > 0.0 THEN
399         TrimDecel_Pps2 := DecelTrim * 250.0 / MmPerPulse;
400     ELSE
```

```
401         TrimDecel_Pps2 := SlowDecel_Pps2;
402       END_IF;
403       SpdBand_Pps := SpdDelta_Percent * 1000.0 / 60.0 / MmPerPulse;
404
405       (* v6d: speeds from the potentiometers, valid in auto AND ManMode     *)
406       (*      (m/min = Pot / Input_Ratio, same as Motor_Manager rung 8)     *)
407       IF (Input_Ratio > 0.0) AND ManMode THEN
408         (* v6i: ManMode drives the inverter with the raw pots (rung 17) *)
409         SpdFinal_Pps := INT_TO_REAL(Pot2_Slow_Velocity) / Input_Ratio * 1000.0 /
    60.0 / MmPerPulse;
410         SpdWork_Pps  := INT_TO_REAL(Pot1_Work_Velocity) / Input_Ratio * 1000.0 /
    60.0 / MmPerPulse;
411       ELSIF Input_Ratio > 0.0 THEN
412         SpdFinal_Pps := INT_TO_REAL(Pot2_Mem) / Input_Ratio * 1000.0 / 60.0 /
    MmPerPulse;
413         SpdWork_Pps  := INT_TO_REAL(Pot1_Mem) / Input_Ratio * 1000.0 / 60.0 /
    MmPerPulse;
414       ELSE
415         SpdFinal_Pps := LowSpd_Display * 1000.0 / 60.0 / MmPerPulse;
416         SpdWork_Pps  := Speed_Display  * 1000.0 / 60.0 / MmPerPulse;
417       END_IF;
418       (* v6y: rewinder AUTO stop -> work speed limited to RwAutoMaxSpdAct *)
419       (* (limit + value taken over only at standstill, Motor_Manager rung 5) *)
420       IF RwAutoLimAct AND NOT ManMode
421         AND (SpdWork_Pps > RwAutoMaxSpdAct * 1000.0 / 60.0 / MmPerPulse) THEN
422         SpdWork_Pps := RwAutoMaxSpdAct * 1000.0 / 60.0 / MmPerPulse;
423       END_IF;
424       SpdRef_Pps := MAX(SpdAct_Pps, SpdWork_Pps);
425
426       IF RwStopDecel > 0.0 THEN
427         StopLenFinal_Pls := SpdFinal_Pps * RwTreact
428                           + (SpdFinal_Pps * SpdFinal_Pps) / (2.0 * RwStopDecel);
429       ELSE
430         StopLenFinal_Pls := SpdFinal_Pps * (RwTreact + RwTstopDef / 2.0);
431       END_IF;
432
433       SlowLenBase_Pls := StopLenFinal_Pls * RwTrim;
434       IF (SpdRef_Pps > SpdFinal_Pps) AND (SlowDecel_Pps2 > 0.0) THEN
435         IF SlowOneRamp OR (TrimDecel_Pps2 <= 0.0) THEN
436           (* manual or learned decel: one constant ramp *)
437           SlowLenBase_Pls := SlowLenBase_Pls
438             + (SpdRef_Pps * SpdRef_Pps - SpdFinal_Pps * SpdFinal_Pps)
439              / (2.0 * SlowDecel_Pps2);
440         ELSE
441           (* auto: coarse ramp down to the band, trim ramp inside it *)
442           SpdKnee_Pps := SpdFinal_Pps + SpdBand_Pps;
443           IF SpdRef_Pps > SpdKnee_Pps THEN
444             SlowLenBase_Pls := SlowLenBase_Pls
445               + (SpdRef_Pps * SpdRef_Pps - SpdKnee_Pps * SpdKnee_Pps)
446                / (2.0 * SlowDecel_Pps2)
447               + (SpdKnee_Pps * SpdKnee_Pps - SpdFinal_Pps * SpdFinal_Pps)
448                / (2.0 * TrimDecel_Pps2);
449           ELSE
450             SlowLenBase_Pls := SlowLenBase_Pls
451               + (SpdRef_Pps * SpdRef_Pps - SpdFinal_Pps * SpdFinal_Pps)
452                / (2.0 * TrimDecel_Pps2);
453           END_IF;
454         END_IF;
```

```
455     END_IF;
456
457     RwSlowLen_Pls := SlowLenBase_Pls + RwSlowMargin_m * 1000.0 / MmPerPulse;
458
459     IF UwSlowMargin_m > 0.0 THEN
460         UwMarginUsed_m := UwSlowMargin_m;
461     ELSE
462         UwMarginUsed_m := RwSlowMargin_m;
463     END_IF;
464     UwSlowLen_Pls := SlowLenBase_Pls + UwMarginUsed_m * 1000.0 / MmPerPulse;
465 END_IF;
466
467 IF RwDiam >= RwTargetDiam THEN
468     RwRemain_Pls := 0.0;
469 ELSIF RwMat_K > 0.0 THEN
470     RwRemain_Pls := (RwTargetDiam * RwTargetDiam - RwDiam * RwDiam)
471                     / (2.0 * RwMat_K);
472 ELSE
473     RwRemain_Pls := 1.0E9;                    (* K not learned: plain target *)
474 END_IF;
475
476 RwRemain_m  := RwRemain_Pls  * MmPerPulse / 1000.0;
477 RwStopLen_m := StopLen_Pls   * MmPerPulse / 1000.0;
478 RwSlowLen_m := RwSlowLen_Pls * MmPerPulse / 1000.0;
479
480 (* ---- 6b. Live operator displays ---------------------------------------- *)
481 (* Rewinder: diameter where slow-down / stop command will happen          *)
482 IF RwMat_K > 0.0 THEN
483     SqrtArg := RwTargetDiam * RwTargetDiam - 2.0 * RwMat_K * RwSlowLen_Pls;
484     IF SqrtArg > 0.0 THEN
485         RwSlowAtDiam := SQRT(SqrtArg);
486     ELSE
487         RwSlowAtDiam := 0.0;
488     END_IF;
489     SqrtArg := RwTargetDiam * RwTargetDiam - 2.0 * RwMat_K * StopLen_Pls;
490     IF SqrtArg > 0.0 THEN
491         RwStopAtDiam := SQRT(SqrtArg);
492     ELSE
493         RwStopAtDiam := 0.0;
494     END_IF;
495 ELSE
496     (* v6t: thickness not learned: fixed slow-down point before the target *)
497     IF RwUseSlow AND (RwSlowFallbackMm > 0.0) THEN
498         RwSlowAtDiam := RwTargetDiam - RwSlowFallbackMm;
499     ELSE
500         RwSlowAtDiam := RwTargetDiam;
501     END_IF;
502     RwStopAtDiam := RwTargetDiam;
503 END_IF;
504
505 (* Unwinder: meter count where the slow-down will start                   *)
506 IF MmPerPulse > 0.0 THEN
507     UwSlow_at_Metre := UwStop_At_Meter - UwSlowLen_Pls * MmPerPulse / 1000.0;
508 ELSE
509     UwSlow_at_Metre := UwStop_At_Meter;
510 END_IF;
511 IF UwSlow_at_Metre < 0.0 THEN
512     UwSlow_at_Metre := 0.0;
513 END_IF;
```

```
514
515 (* Unwinder slow-down request, latched                    *)
516 IF (Current_Metre < UwSlow_at_Metre - 50.0) OR (UwStop_At_Meter <= 0.0) THEN
517     UwSlowReq := FALSE;                    (* counter reset / longer roll *)
518 END_IF;
519 IF (UwStop_At_Meter > 0.0) AND (Current_Metre >= UwSlow_at_Metre) THEN
520     UwSlowReq := TRUE;
521 END_IF;
522
523 (* ---- 7. Rewinder requests: latched, reset when a new core is loaded ---- *)
524 IF NOT RwStopEnable
525    OR ((RwDiam < RwTargetDiam - RwResetHyst) AND NOT RwSensFault) THEN
526     RwStopReq := FALSE;
527     RwSlowReq := FALSE;
528 END_IF;
529
530 IF RwStopEnable THEN
531     IF (RwRemain_Pls <= StopLen_Pls) OR (RwSensFault AND RwStopOnFault) THEN
532         RwStopReq := TRUE;
533     END_IF;
534     IF RwUseSlow AND (RwRemain_Pls <= RwSlowLen_Pls) THEN
535         RwSlowReq := TRUE;
536     END_IF;
537     (* v6t: thickness not learned (RwMat_K = 0): slow down at a fixed     *)
538     (* distance before the target instead of stopping from full speed.   *)
539     IF RwUseSlow AND (RwMat_K <= 0.0) AND (RwSlowFallbackMm > 0.0)
540        AND NOT RwSensFault
541        AND (RwDiam >= RwTargetDiam - RwSlowFallbackMm) THEN
542         RwSlowReq := TRUE;
543     END_IF;
544 END_IF;
545
546 (* ---- 8. One AUTO at a time + single "SLOW DOWN AT" display ------------- *)
547 IF UwStopEnable AND NOT UwEnMem THEN              (* unwinder AUTO switched
    ON *)
548     RwStopEnable := FALSE;
549 END_IF;
550 IF RwStopEnable AND NOT RwEnMem THEN              (* rewinder AUTO switched
    ON *)
551     UwStopEnable := FALSE;
552 END_IF;
553 UwEnMem := UwStopEnable;
554 RwEnMem := RwStopEnable;
555
556 IF RwStopEnable THEN
557     SlowAtDisp := RwSlowAtDiam;                (* mm *)
558     SlowAtMode := 2;
559 ELSIF UwStopEnable THEN
560     SlowAtDisp := UwSlow_at_Metre;             (* m  *)
561     SlowAtMode := 1;
562 ELSE
563     SlowAtDisp := 0.0;
564     SlowAtMode := 0;                           (* hide display *)
565 END_IF;
566 SlowAtShow := (SlowAtMode <> 0);
567
568 (* ---- 9. Automatic stop master = selector 12SA5 (I_AutoStop_OS) -------- *)
569 AutoSlowCmd := I_AutoStop_OS AND
570             ((UwStopEnable AND UwSlowReq) OR (RwStopEnable AND RwSlowReq));
```

```
571
572  AutoStopCmd := I_AutoStop_OS AND
573              ((UwStopEnable AND (UwStop_At_Meter > 0.0)
574                AND (Current_Metre >= UwStop_At_Meter))
575               OR (RwStopEnable AND RwStopReq));
576
577 (* ---- 10. Unwinder roll empty (for alarm 15) ---------------------------- *)
578 IF UwDiam_Act > (UwDiamStop + UwEmptyHyst) THEN
579     UnWinder_Empty := FALSE;                    (* new roll loaded *)
580  END_IF;
581 IF (UwDiam_Act > 0.0) AND (UwDiam_Act <= UwDiamStop) THEN
582     UnWinder_Empty := TRUE;                     (* roll empty *)
583  END_IF;
584
585 (* v6x: new parent roll loaded (empty -> not empty) -> HMI popup        *)
586 (* "DID YOU CHANGE THE PAPER (GSM)?". YES = forget the learned thickness *)
587 (* (same as NEW PAPER). NO = the HMI resets GsmAsk.                      *)
588 IF UwEmptyMem AND NOT UnWinder_Empty AND Measure_ON THEN
589     GsmAsk := TRUE;
590  END_IF;
591  UwEmptyMem := UnWinder_Empty;
592 IF GsmAskYes THEN
593     RwMat_K := 0.0;
594     LearnFailCnt := 0;
595     MatChangeCnt := 0;
596     GsmAskYes := FALSE;
597     GsmAsk := FALSE;
598  END_IF;
599
600 (* ---- 11. Rewinder max diameter protection - ALWAYS active ------------- *)
601 (* Independent of selector 12SA5 and of the AUTO buttons.                *)
602 (* Mechanical limit 680 mm radius = 1360 mm                             *)
603 IF NOT RwSensFault THEN
604     RwLastValidDiam := RwDiam;
605  END_IF;
606
607 IF RwDiam >= RwMaxDiam THEN
608     RwRemainMax_Pls := 0.0;
609  ELSIF RwMat_K > 0.0 THEN
610     RwRemainMax_Pls := (RwMaxDiam * RwMaxDiam - RwDiam * RwDiam)
611                        / (2.0 * RwMat_K);
612  ELSE
613     RwRemainMax_Pls := 1.0E9;                   (* K not learned: plain limit *)
614  END_IF;
615
616 (* Display: diameter where the max-protection slow-down will start       *)
617 IF RwMat_K > 0.0 THEN
618     SqrtArg := RwMaxDiam * RwMaxDiam - 2.0 * RwMat_K * RwSlowLen_Pls;
619     IF SqrtArg > 0.0 THEN
620         RwMaxSlowAtDiam := SQRT(SqrtArg);
621     ELSE
622         RwMaxSlowAtDiam := 0.0;
623     END_IF;
624  ELSIF RwMaxSlowFallback > 0.0 THEN
625     RwMaxSlowAtDiam := RwMaxSlowFallback;       (* fallback in use *)
626  ELSE
627     RwMaxSlowAtDiam := RwMaxDiam;
628  END_IF;
629
```

```
(* Reset only after the roll is removed (diameter well below the limit)   *)
IF (RwDiam < RwMaxDiam - RwResetHyst) AND NOT RwSensFault THEN
    RwMaxStop := FALSE;
    RwMaxSlow := FALSE;
END_IF;

IF NOT RwSensFault THEN
    IF RwRemainMax_Pls <= StopLen_Pls THEN
        RwMaxStop := TRUE;                    (* speed-compensated stop *)
    END_IF;
    IF RwRemainMax_Pls <= RwSlowLen_Pls THEN
        RwMaxSlow := TRUE;                    (* warning slow-down + horn *)
    END_IF;
    IF (RwMat_K <= 0.0) AND (RwMaxSlowFallback > 0.0)
      AND (RwDiam >= RwMaxSlowFallback) THEN
        RwMaxSlow := TRUE;                    (* fallback while K not learned *)
    END_IF;
ELSIF (AI_Diam_Rewinder < RwRawMin) AND (RwLastValidDiam > RwMaxDiam -
100.0) THEN
    RwMaxStop := TRUE;                        (* roll grew into the DT35 near zone *)
END_IF;

(* Max diameter protection added OUTSIDE the 12SA5 master (section 9)     *)
AutoSlowCmd := AutoSlowCmd OR RwMaxSlow;
AutoStopCmd := AutoStopCmd OR RwMaxStop;

(* ---- 12. Learning supervision -> alarm 16 (section Alarms) ------------ *)
(* Learned paper thickness, for display / plausibility check              *)
IF MmPerPulse > 0.0 THEN
    RwThick_mm := RwMat_K * 3.14159 / (2.0 * MmPerPulse);
ELSE
    RwThick_mm := 0.0;
END_IF;

(* Fault: too many consecutive windows / stops that should have           *)
(* learned but did not. Clears by itself on the next good result.         *)
(* v6w: ALARM RESET (panel button or HMI) also clears the fail counters, *)
(* so alarm 16 resets in one press. It comes back only if the learning   *)
(* fails again RwLearnFailMax / RwStopFailMax times in a row.             *)
IF I_RsetAlarm OR Alarm_Reset THEN
    LearnFailCnt := 0;
    StopFailCnt := 0;
END_IF;
GSM_LearnFault := (LearnFailCnt >= RwLearnFailMax)
          OR (StopFailCnt >= RwStopFailMax);      (* -> alarm 16 *)

(* v6e: learning status for the HMI. FALSE until learned.                 *)
GSM_Learned_OK := (RwMat_K > 0.0)
              AND ((RwStopDecel > 0.0) OR NOT RwLearnStop)
              AND NOT GSM_LearnFault;

(* ---- 13. v6h: keep T-one in automatic tension control during the stop - *)
(* While running: BrakeHold = Brake_Cmd (no change in running behaviour).  *)
(* After the stop: stays ON until the web is below 5 m/min, after 15 s.    *)
(* or on E-stop. Already TRUE on the stop scan (Output_Manager runs before *)
(* this section), so O_Brake never drops for one scan.                    *)
(* Output_Manager rung 2:  O_Brake := Brake_Cmd OR BrakeHold              *)
IF O_MotorStart THEN
    BrakeHold := Brake_Cmd AND ControlVoltage;
```

```
688     BrakeHoldTicks := 0;
689   ELSIF BrakeHold THEN
690     IF Tick THEN
691       BrakeHoldTicks := BrakeHoldTicks + 1;
692     END_IF;
693     (* v6v: signed web speed: a web pulled BACK (negative speed) also     *)
694     (* releases the T-one (trace 10/28 15:00: -6.6 m/min, about 20 cm).  *)
695     IF NOT ControlVoltage OR (Velocity_M_min < 5.0) OR (BrakeHoldTicks >= 150)
    THEN
696       BrakeHold := FALSE;
697     END_IF;
698   END_IF;
699
700 (* ---- 14. v6j: TEACH CORE - automatic offset with an empty core -------- *)
701 (* HMI: set RwTeachCore (button, Set ON). Conditions: motor stopped, web  *)
702 (* not moving, DT35 OK. After 2 s stable (filter settled):                *)
703 (*    RwAxisDist := RwDist + RwCoreDiameter / 2   -> RwDiam = RwCoreDiameter
    *)
704 (* RwTeachState: 0 idle, 1 teaching (wait 2 s), 2 done OK,                *)
705 (*               3 refused (motor running / web moving / DT35 fault).     *)
706 (* Only the offset is taught: the scale is RwDist10V - RwDist0V           *)
707 (* (= real distance between the two DT35 taught points).                  *)
708 IF RwTeachCore THEN
709     IF O_MotorStart OR (RwSpeed > 1.0) OR RwSensFault
710       OR NOT RwFiltInit OR (RwCoreDiameter <= 0.0) THEN
711       RwTeachState := 3;                    (* refused *)
712       RwTeachCore := FALSE;
713       TeachTicks := 0;
714     ELSE
715       RwTeachState := 1;                    (* teaching *)
716       IF Tick THEN
717         TeachTicks := TeachTicks + 1;
718       END_IF;
719       IF TeachTicks >= 20 THEN              (* 2 s stable *)
720         RwAxisDist_Prev := RwAxisDist;      (* v6l: keep previous *)
721         RwAxisDist := RwDist + RwCoreDiameter / 2.0;
722         RwTeachState := 2;                  (* done *)
723         RwTeachCore := FALSE;
724         TeachTicks := 0;
725       END_IF;
726     END_IF;
727   ELSE
728     TeachTicks := 0;
729     IF O_MotorStart THEN
730       RwTeachState := 0;                    (* clear status on next run *)
731     END_IF;
732   END_IF;
733
734 (* ---- 14b. v6k: unwinder TEACH CORE (HMI button Core_Measure) ---------- *)
735 (* Same behaviour as section 14. Sonar is not filtered, so the 2 s are   *)
736 (* averaged:  UwCoreDistance := average(UwDiam_Measure)  (sonar -> core   *)
737 (* surface). UwDiam_Act = (UwCoreDistance - UwDiam_Measure) * 2           *)
738 (*                      + UwCoreDiameter   (General rung 14, unchanged). *)
739 (* UwTeachState: 0 idle, 1 teaching, 2 done OK, 3 refused.                *)
740 IF Core_Measure THEN
741     IF O_MotorStart OR (RwSpeed > 1.0) OR (UwCoreDiameter <= 0.0)
742       OR (UwDiam_Measure <= 0.0) THEN
743       UwTeachState := 3;                    (* refused *)
744       Core_Measure := FALSE;
```

```
        UwTeachTicks := 0;
        UwTeachSum := 0.0;
    ELSE
        UwTeachState := 1;                  (* teaching *)
        IF Tick THEN
            UwTeachTicks := UwTeachTicks + 1;
            UwTeachSum := UwTeachSum + UwDiam_Measure;
        END_IF;
        IF UwTeachTicks >= 20 THEN          (* 2 s, 20 samples *)
            UwCoreDist_Prev := UwCoreDistance;    (* v6l: keep previous *)
            UwCoreDistance := UwTeachSum / INT_TO_REAL(UwTeachTicks);
            UwTeachState := 2;              (* done *)
            Core_Measure := FALSE;
            UwTeachTicks := 0;
            UwTeachSum := 0.0;
        END_IF;
    END_IF;
ELSE
    UwTeachTicks := 0;
    UwTeachSum := 0.0;
    IF O_MotorStart THEN
        UwTeachState := 0;                  (* clear status on next run *)
    END_IF;
END_IF;

(* ---- 14c. v6l: UNDO teach = swap current <-> previous (machine stopped) *)
(* Pressing UNDO twice goes back to the taught value. State 4 = RESTORED. *)
IF RwTeachUndo THEN
    IF NOT O_MotorStart AND (RwAxisDist_Prev > 0.0) THEN
        TeachSwap := RwAxisDist;
        RwAxisDist := RwAxisDist_Prev;
        RwAxisDist_Prev := TeachSwap;
        RwTeachState := 4;                  (* restored *)
    ELSE
        RwTeachState := 3;                  (* refused *)
    END_IF;
    RwTeachUndo := FALSE;
END_IF;
IF UwTeachUndo THEN
    IF NOT O_MotorStart AND (UwCoreDist_Prev > 0.0) THEN
        TeachSwap := UwCoreDistance;
        UwCoreDistance := UwCoreDist_Prev;
        UwCoreDist_Prev := TeachSwap;
        UwTeachState := 4;                  (* restored *)
    ELSE
        UwTeachState := 3;                  (* refused *)
    END_IF;
    UwTeachUndo := FALSE;
END_IF;
```

1-9.Data

1-9-1.Data Types

Name	Type	Comment

1-9-2.Global Variables

Name	Data Type	Initial Value	AT	Retain	Constant	Network Publish	Comment
VAR_GLOBAL							
Pot1_Work_Velocity	INT		BuiltInIO://opt/#0/Ch1_Analog_Input_Value	False	False	Publish Only	Potentiometer 1 value - Work speed -
InvRef	INT		BuiltInIO://opt/#0/Ch2_Analog_Output_Value	False	False	Publish Only	Inverter speed reference
Fix10Vdc	INT		BuiltInIO://opt/#0/Ch1_Analog_Output_Value	False	False	Publish Only	Fixed 10 Vdc generator
VelMax	REAL	800		True	False	Publish Only	Maximum speed
VelSet	REAL			False	False	Publish Only	Set speed
RwSlowDecelK	REAL	0.15		True	False	Publish Only	Learned slow-down decel per mm of rewinder diameter (mm/s2 per mm)
RwMaxSlowFallback	REAL	1360.0		True	False	Publish Only	Rewinder max: slow-down diameter while K not learned (mm)
RwThickMin_mm	REAL	0.02		True	False	Publish Only	Learning: min plausible paper thickness (mm)
RwThickMax_mm	REAL	2.0		True	False	Publish Only	Learning: max plausible paper thickness (mm)
RwDecelMin_mm	REAL	20.0		True	False	Publish Only	Learning: min plausible stop decel (mm/s2)
RwDecelMax_mm	REAL	5000.0		True	False	Publish Only	Learning: max plausible stop decel (mm/s2)
RwLearnFailMax	INT	12		True	False	Publish Only	Learning: failed windows before alarm (12 = 2 min)
RwStopFailMax	INT	3		True	False	Publish Only	Learning: failed stops before alarm
RwThick_mm	REAL			False	False	Publish Only	Learned paper thickness (mm)
OverSpeed	BOOL			False	False	Publish Only	Speed too high
WorkSpd_Max	REAL	400.0		True	False	Publish Only	Work speed pot full scale (m/min), 50..VelMax
SoftStartTime	REAL	3.0		True	False	Publish Only	AUTO start S-curve: time for the accel ramp to rise from 5 % to 100 % (s), 0 = off
WorkSpd_Max_Set	REAL	400.0		True	False	Publish Only	HMI request: work pot full scale (m/min), applied only with motor stopped
RwMatReset	BOOL	False		False	False	Publish Only	HMI NEW PAPER: forget the learned paper thickness RwMat_K (PLC resets it)
FinalSpd_Max_Set	REAL	40.0		True	False	Publish Only	HMI request: final pot full scale (m/min), applied only with motor stopped
FinalSpd_Max	REAL	40.0		True	False	Publish Only	Final speed pot full scale (m/min), 5..WorkSpd_Max
UnderSpeed	BOOL			False	False	Publish Only	Speed too low
RwLearnReset	BOOL	False		False	False	Publish Only	HMI / Watch: clear learned decels RwStopDecelK, RwStopDecel, RwSlowDecelK (PLC resets it)

RwCoreDiameter	REAL	100.0		True	False	Publish Only	Rewinder core outside diameter for TEACH CORE (mm)
RwTeachCore	BOOL	False		False	False	Publish Only	HMI: TEACH CORE request (PLC resets it)
RwTeachState	INT	0		False	False	Publish Only	TEACH CORE: 0 idle 1 teaching 2 done 3 refused
SpdDelta_Percent	REAL			True	False	Publish Only	Speed difference percentage
Encoder_Pos	DINT		IOBus://unit#3/Ch1 Encoder Present Position	False	False	Publish Only	Encoder counter
UwTeachState	INT	0		False	False	Publish Only	Unwinder TEACH CORE: 0 idle 1 teaching 2 done 3 refused
PulseRate	UDINT		IOBus://unit#3/Ch1 Pulse Rate	False	False	Publish Only	Pulses per 100 ms
Encoder_On	BOOL		IOBus://unit#3/Ch1 Encoder Counter Operation Command/Ch1 Counter Enable	False	False	Publish Only	Encoder enable
Encoder_Reset	BOOL		IOBus://unit#3/Ch1 Encoder Counter Operation Command/Ch1 Internal Reset Execution	False	False	Publish Only	Encoder reset
HMI_Screen	ARRAY[0..1] OF DINT			False	False	Publish Only	HMI panel Screen
Velocity_M_min	REAL			True	False	Publish Only	Speed in m/min
Current_Metre	REAL			True	False	Publish Only	Current metres
I_RsetCnt	BOOL			False	False	Publish Only	Metre counter reset button
Speed_Display	REAL			True	False	Publish Only	Set speed (displayed)
I_ControlVoltage	BOOL		BuiltInIO://cpu/#0/Input_Bit_00	False	False	Publish Only	Auxiliaries ON
I_Start_PB	BOOL		BuiltInIO://cpu/#0/Input_Bit_01	False	False	Publish Only	Start button
I_Stop_PB	BOOL		BuiltInIO://cpu/#0/Input_Bit_02	False	False	Publish Only	Stop button
I_Motor_Thermal	BOOL		BuiltInIO://cpu/#0/Input_Bit_03	False	False	Publish Only	Motor thermal switch
I_Prot_ByPass	BOOL		BuiltInIO://cpu/#0/Input_Bit_04	False	False	Publish Only	Guards-bypassed selector switch - Under Panel
I_Hub_Positi	BOOL		BuiltInIO://c	False	False	Publish Only	Hub/chuck in extraction pos

on			pu/#0/Input_Bit_05				ition
I_FrontGuard_Closed	BOOL		BuiltInIO://cpu/#0/Input_Bit_06	False	False	Publish Only	Front guard closed
I_LowSpd_OS	BOOL		BuiltInIO://cpu/#0/Input_Bit_07	False	False	Publish Only	Slow speed button
I_VFD_Alarm	BOOL		BuiltInIO://cpu/#0/Input_Bit_08	False	False	Publish Only	Inverter in alarm
I_SafeGuard_Closed	BOOL			False	False	Publish Only	Side doors closed
I_OverRider_PH	BOOL		BuiltInIO://cpu/#0/Input_Bit_13	False	False	Publish Only	Roll diameter photocell
I_OR_Position_Sens	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN01 (Pin 1.1) _6000_01	False	False	Publish Only	OverRider position sens
I_OR_Pressure_Sens	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN02 (Pin 2.1) _6000_02	False	False	Publish Only	OverRider pressure switch
I_OR_Auto_OS	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN03 (Pin 1.2) _6000_03	False	False	Publish Only	OverRider in automatic
I_OR_Cont_OS	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN04 (Pin 2.2) _6000_04	False	False	Publish Only	OverRider in continuous
O_MotorStart	BOOL		BuiltInIO://cpu/#0/Output_Bit_00	False	False	Publish Only	Motor start
I_FanTemp_Ok	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN06 (Pin 2.3) _6000_06	False	False	Publish Only	Motor (fan) thermal OK
O_Alarm_Array	BOOL		ECAT://node#[1,1]/IB IL 24 DO 8/HD-ECO Output_OUT01 (Pin 1.1) _7000_01	False	False	Publish Only	Alarm lamp
O_Prot_ByPass_Lamp	BOOL			False	False	Publish Only	Guards bypassed lamp

O_Motor_Lamp	BOOL		BuiltInIO://cpu/#0/Output_Bit_01	False	False	Publish Only	Motor ON lamp
O_RwUnload	BOOL		ECAT://node #[1,1]/IB IL 24 DO 8/HD-ECO Output_OUT 03 (Pin 1.2) _7000_03	False	False	Publish Only	Unloaders enable - opening
O_FrontGuard_ByPass	BOOL		ECAT://node #[1,1]/IB IL 24 DO 8/HD-ECO Output_OUT 04 (Pin 2.2) _7000_04	False	False	Publish Only	Front guard bypass
O_Brake	BOOL		BuiltInIO://cpu/#0/Output_Bit_02	False	False	Publish Only	Brake command
O_OR_Auto_Lamp	BOOL		BuiltInIO://cpu/#0/Output_Bit_06	False	False	Publish Only	OverRider auto/continuous lamp
O_OverRider_Up	BOOL		BuiltInIO://cpu/#0/Output_Bit_09	False	False	Publish Only	Raise OverRider ~10mm Over Rewinder roll
O_OverRider_Hold	BOOL		BuiltInIO://cpu/#0/Output_Bit_04	False	False	Publish Only	OverRider Lock
O_CoolingFan	BOOL		BuiltInIO://cpu/#0/Output_Bit_03	False	False	Publish Only	Brake and motor fans
Fan_Timeout	REAL	0.5		True	False	Publish Only	Fans stop timeout [min]
OverRider_TimeOFF	REAL	0.01		True	False	Publish Only	OverRider photocell OFF timeout (panel)
OverRider_TimeON	REAL	0.05		True	False	Publish Only	OverRider photocell ON timeout (panel)
AlarmOn	BOOL			False	False	Publish Only	Alarm active
BuzzerOn	BOOL			False	False	Publish Only	Panel buzzer command
Msg	INT			False	False	Publish Only	Message
Index_B	INT			False	False	Publish Only	Alarm index
MotStop	BOOL			False	False	Publish Only	Motor stopped
Run_Normal_Spd	BOOL			False	False	Publish Only	Motor running - normal speed
Clock1s	BOOL			False	False	Publish Only	1 s clock
ManMode	BOOL			True	False	Publish Only	Manual mode
Speed_Man_SP	INT			True	False	Publish Only	% speed in manual
Counter_Reset	BOOL			False	False	Publish Only	Counter reset from panel
I_FrontGuard_Open	BOOL		BuiltInIO://cpu/#0/Input_Bit_09	False	False	Publish Only	Front guard open
RwUnload_Enable	BOOL		BuiltInIO://cpu/#0/Output_Bit_07	False	False	Publish Only	Unloaders enable - closing
JogSpd_SP	REAL	10.0		True	False	Publish Only	Speed in manual
O_AccDec2	BOOL		BuiltInIO://cpu/#0/Output_Bit_05	False	False	Publish Only	Second acc/dec ramp
AccelTrim	REAL			True	False	Publish Only	Acceleration coefficient in run

DecelTrim	REAL			True	False	Publish Only	Deceleration coefficient in run
DecelCoarse	REAL			True	False	Publish Only	Acceleration coefficient at start (orig. comment; name suggests decel)
AccelCoarse	REAL			True	False	Publish Only	Acceleration coefficient at start (orig. comment; name suggests accel)
O_AcousticSignal	BOOL		BuiltInIO://cpu/#0/Output_Bit_08	False	False	Publish Only	Horn / Acoustic signal
Pot2_Slow_Velocity	INT		BuiltInIO://opt/#0/Ch2_Analog_Input_Value	False	False	Publish Only	Potentiometer 2 value - Final speed -
I_AutoStop_OS	BOOL		BuiltInIO://cpu/#0/Input_Bit_10	False	False	Publish Only	Stop on metre counter
I_RollEnd_SlowDown	BOOL		BuiltInIO://cpu/#0/Input_Bit_11	False	False	Publish Only	Slow down at end of roll
I_WebBreak_Monitor	BOOL		BuiltInIO://cpu/#0/Input_Bit_12	False	False	Publish Only	Paper break monitoring ON
I_Vacuum_OK	BOOL		IOBus://unit#3/Ch1 Reset/External Input Status/Ch1 External Input 0 Status	False	False	Publish Only	Extractor fan thermal OK
I_SlowSens_PH	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN05 (Pin 1.3) _6000_05	False	False	Publish Only	Slow-down photocell
I_WebBreak_PH	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN07 (Pin 1.4) _6000_07	False	False	Publish Only	Paper break photocell
UwStop_At_Meter	REAL	5100		True	False	Publish Only	Unwinder Stop at metres
UwSlowMargin_m	REAL	10		True	False	Publish Only	Unwinder Slow-down metres
LowSpd_Display	REAL			True	False	Publish Only	Slow-down speed (displayed)
RwMaxDiam	REAL	1380.0		True	False	Publish Only	Rewinder max diameter protection (mm)
RwMaxStop	BOOL			False	False	Publish Only	Rewinder max diameter STOP (latched)
Encoder_Pulse	INT	1000		True	False	Publish Only	Encoder pulses/Rotation
RwSlowDecelLrn	REAL	0.0		True	False	Publish Only	Learned real slow-down decel (mm/s2) - 0 = not learned
LowSpd_Ma	INT			False	False	Publish Only	% slow-down speed in man

n_SP							ual
CountReset_Req	BOOL			False	False	Publish Only	Counter reset requested
Psw	INT			False	False	Publish Only	Password
Language	DINT			True	False	Publish Only	Language
I_RsetAlarm	BOOL		ECAT://node #[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN08 (Pin 2.4) _6000_08	False	False	Publish Only	Alarm reset
Fix10Vdc_1	UINT			False	False	Publish Only	Fixed 10 Vdc generator
EncoderWheel_Diam	REAL	400		True	False	Publish Only	Encoder wheel diameter
AlarmCodeSV	ARRAY[0..32] OF BOOL			False	False	Publish Only	Panel alarms
SafeGuard_ByPass	BOOL			False	False	Publish Only	Guards bypassed
FrontGuard_Closed	BOOL			False	False	Publish Only	Front guard closed
Anal3	UINT			False	False	Publish Only	(?) Analog 3
Anal4	UINT			False	False	Publish Only	(?) Analog 4
SoftStop	BOOL	False		False	False	Publish Only	Soft stop in progress (HMI)
SoftStopEnable	BOOL	True		True	False	Publish Only	STOP button = soft stop (False = immediate as before)
SoftStopEndSpd	REAL	5.0		True	False	Publish Only	Soft stop: run command drops below this speed (m/min)
UwDiam_Measure	REAL			True	False	Publish Only	Measured distance
UwCoreDiameter	REAL	100		True	False	Publish Only	Core Outside diameter [mm]
Core_Measure	BOOL	False		True	False	Publish Only	Stored core value
UwCoreDistance	REAL	785.56		True	False	Publish Only	Sonar distance to empty core. [mm]
AI_Diam_Unwinder	INT		IOBus://unit#2/Ch1 Analog Input Value	False	False	Publish Only	Analog value - UnWinder Diameter
Measure_ON	BOOL	True		True	False	Publish Only	Measurement enabled
UwDiam_Act	REAL			True	False	Publish Only	Measured roll diameter
UwSlowDown_Diam	REAL	200		True	False	Publish Only	Unwinder Slow-down diameter [mm]
UwDiamStop	REAL	110		True	False	Publish Only	Stop diameter
Alarm_Reset	BOOL			False	False	Publish Only	Alarm reset from panel
Psw22	BOOL			False	False	Publish Only	Password 22
Psw44	BOOL			False	False	Publish Only	Password 44
I_Roll_OK	BOOL			False	False	Publish Only	Roll OK / Broken Holder Joint
I_RollEnd_PH	BOOL			False	False	Publish Only	End-of-roll photocell
AI_Diam_Rewinder	INT		IOBus://unit#2/Ch2 Analog Input Value	False	False	Publish Only	Analog value - ReWinder Diameter
RwStopEnable	BOOL	False		True	False	Publish Only	Rewinder diameter stop enabled
RwTargetDia	REAL	300.0		True	False	Publish Only	Rewinder final diameter (m

m							m)
RwUseSlow	BOOL	True		True	False	Publish Only	Slow down before target
RwSlowMargin_m	REAL	10.0		True	False	Publish Only	Rewinder Metres at slow speed before stop
RwSlowDecel	REAL	0.0		True	False	Publish Only	Rewinder Slow-down decel mm/s2 - 0 = auto from DecStart
RwResetHyst	REAL	100.0		True	False	Publish Only	Reset requests below target - this [mm]
RwDist0V	REAL	0.0		True	False	Publish Only	DT35 distance at 0 V (mm)
RwDist10V	REAL	750.0		True	False	Publish Only	DT35 distance at 10 V (mm)
RwRawMin	INT	10		True	False	Publish Only	Raw below = sensor fault
RwRawMax	INT	3900		True	False	Publish Only	Raw above = sensor fault
RwStopOnFault	BOOL	True		False	False	Publish Only	Stop on sensor fault
RwAxisDist	REAL	675.3		True	False	Publish Only	Sensor to shaft centre (mm) - teach
RwFilter_K	REAL	0.2		True	False	Publish Only	Distance filter per 100 ms (0..1)
RwWinTicks	INT	100		True	False	Publish Only	Material learn window (100 = 10 s)
RwMinPulses	DINT	1000		True	False	Publish Only	Min pulses in window to learn
RwLearn_K	REAL	0.3		True	False	Publish Only	Material factor filter weight
RwMat_K	REAL	0.0		True	False	Publish Only	Learned material factor
RwLearnStop	BOOL	True		True	False	Publish Only	Learn stop decel on every stop
RwMinLearnSpd	REAL	50.0		True	False	Publish Only	Min speed to learn a stop (m/min)
RwStopDecel	REAL	0.0		True	False	Publish Only	Learned stop decel (pulses/s2)
RwTreact	REAL	0.1		True	False	Publish Only	Stop reaction time (s)
RwTstopDef	REAL	3.0		True	False	Publish Only	Stop time until decel learned (s)
RwTrim	REAL	1.0		True	False	Publish Only	Stop length trim (>1 = earlier)
RwDist	REAL			False	False	Publish Only	Filtered distance (mm)
RwDiam	REAL			False	False	Publish Only	Rewinder diameter (mm)
RwSpeed	REAL			False	False	Publish Only	Line speed from encoder (m/min)
RwRemain_m	REAL			False	False	Publish Only	Metres to target diameter
RwStopLen_m	REAL			False	False	Publish Only	Stopping length now (m)
RwSlowLen_m	REAL			False	False	Publish Only	Rewinder Slow-down length [m]
RwStopReq	BOOL			False	False	Publish Only	Rewinder diameter STOP request
RwSlowReq	BOOL			False	False	Publish Only	Rewinder diameter SLOW request
RwSensFault	BOOL			False	False	Publish Only	Rewinder DT35 sensor fault
UwStopEnable	BOOL	False		True	False	Publish Only	Unwnder Auto Stop - Enable
SlowAtDisp	REAL			False	False	Publish Only	HMI: slow-down starts at (m or mm, see SlowAtMode)
SlowAtMode	INT			False	False	Publish Only	HMI: 0=none 1=unwinder [m] 2=rewinder [mm]
SlowAtShow	BOOL			False	False	Publish Only	HMI: show slow-down display

	UwSlow_at_Metre	REAL			False	False	Publish Only	Unwinder: slow-down starts at [m]
	UwSlowReq	BOOL			False	False	Publish Only	Unwinder metre slow-down request (latched)
	RwSlowAtDiam	REAL			False	False	Publish Only	Rewinder slow-down starts at diameter [mm]
	RwStopAtDiam	REAL			False	False	Publish Only	Rewinder: stop command at diameter (mm)
	RwMaxSlow	BOOL			False	False	Publish Only	Rewinder max diameter SLOW (latched)
	UwEmptyHyst	REAL	5.0		True	False	Publish Only	Unwinder Empty Hysteresis
	GSM_Learned_OK	BOOL			False	False	Publish Only	Paper [Grams / Square Meter] FALSE until GSM learned
	RwMaxSlowAtDiam	REAL	1370.0		False	False	Publish Only	Rewinder max: slow-down starts at diameter (mm)
	RwStopDecelK	REAL	0.0		True	False	Publish Only	Learned stop decel per mm of rewinder diameter (mm/s2 per mm)
	SoftStopDecel	REAL	0.0		True	False	Publish Only	Soft stop decel (mm/s2), 0 = DecelCoarse
	SpdMax_Pending	BOOL	False		False	False	Publish Only	New pot range waiting for the motor to stop
	RwAxisDist_Prev	REAL	675.3		True	False	Publish Only	Rewinder: RwAxisDist before the last TEACH (for UNDO)
	UwCoreDist_Prev	REAL	785.56		True	False	Publish Only	Unwinder: UwCoreDistance before the last TEACH (for UNDO)
	RwTeachUndo	BOOL	False		False	False	Publish Only	HMI: UNDO rewinder TEACH (PLC resets it)
	UwTeachUndo	BOOL	False		False	False	Publish Only	HMI: UNDO unwinder TEACH (PLC resets it)
	Psw33	BOOL			False	False	Publish Only	Password 33
	RwMatChangePct	REAL	25.0		True	False	Publish Only	Paper change detection: window deviation (%), 2 windows in a row = new paper, 0 = off
	RwSlowFallbackMm	REAL	30.0		True	False	Publish Only	Rewinder: slow-down this many mm before the target while the paper thickness is not learned (0 = off)
	RwAutoMaxSpd	REAL	500.0		True	False	Publish Only	Rewinder AUTO stop ON: maximum work speed (m/min), 0 = no limit
	RwWinLen_m	REAL	30.0		True	False	Publish Only	Paper thickness learning: MAX window length (m of paper)
	RwWinGrow_mm	REAL	2.5		True	False	Publish Only	Paper thickness learning: window = this diameter growth (mm), 4 m .. RwWinLen_m
	GsmAsk	BOOL	False		False	False	Publish Only	HMI popup: new parent roll, did you change the paper? (HMI NO resets it)
	GsmAskYes	BOOL	False		False	False	Publish Only	HMI popup YES: forget the learned thickness (PLC resets it)